

Labeled Multi-Key Batched IBE

Guru-Vamsi Policharla
Commonware, Inc.

Abstract

We study Batched IBE through the lens of Multi-Message Signatures – a natural extension of standard digital signatures in which a single signing operation signs an entire vector of messages at once. A public open algorithm then derives an opening for each message, allowing anyone to verify that an *individual* message was signed.

First we show that Multi-Message Signatures are essential by constructing a Multi-Message Signature scheme from every Batched IBE scheme with a sufficiently large plaintext space. We then show how to construct a Batched IBE scheme from a Multi-Message Signature with deterministic verification given an extractable witness encryption for the verification relation, similar to the duality between IBE and digital signatures. Such a witness encryption scheme can be instantiated efficiently for the special case of *linearly-verifiable* pairing-based multi-message signatures.

Next we analyze existing constructions of Batched IBE and extract linearly-verifiable Multi-Message Signatures which provide us with valuable insight into the inner workings of various Batched IBE schemes. Leveraging these insights, we construct the first Batched IBE scheme supporting q key queries per label with an $O(qB)$ CRS size and constant-size ciphertexts.

1 Introduction

Batched identity-based encryption sits at the heart of a recent line of work on batched threshold encryption for encrypted mempools [CGPW25, AFP25, FPTX25, GWWW25].¹ In a batched IBE scheme, encryption is exactly as in ordinary IBE where a ciphertext is addressed to an identity id under a master public key. But key generation is *batched* such that the holder of the master secret key publishes a *succinct* secret key sk_{id} for an entire vector of identities $\text{id} = (\text{id}_1, \dots, \text{id}_B)$, and the same key decrypts every ciphertext addressed to any id_j in the batch while revealing nothing about ciphertexts addressed to identities outside the batch. In the encrypted mempool application the identities are (randomly chosen tags of) pending transactions. Once a block is finalized, the committee releases one short key that opens exactly the B transactions in the block, and every ciphertext left in the mempool remains hidden.

Unfortunately, none of the existing constructions are fully satisfactory for building encrypted mempools. When restricted to constant-size ciphertexts, the schemes of [CGPW25, AFP25, GWWW25] attach a *batch label* (an epoch or block number) to every key and can release at most *one* key per label. The encryptor must therefore guess the label of the batch in which their transaction will be decrypted. If the transaction misses that batch, the ciphertext can never be decrypted and the user must resubmit, hurting the reliability of the system. [FPTX25] removes labels altogether, but its public key only supports q decryptions in *total*, after which the committee must refresh the key with an expensive distributed key generation. Although one can set q large enough that the refresh coincides with the natural committee rotation, the common reference string grows as $O(qB)$. With committees rotating every 7 days and 250 ms slot times, the CRS on BLS12-381 with batch size $B = 8192$ would be a little under 900 GB(!). Although the CRS can be stored on disk and relevant portions of it can be loaded into memory (on a predictable schedule), securely generating and distributing such a large CRS to all validators is still quite tedious.

¹We note that other constructions of batched threshold encryption [BFOQ25, BCF⁺25, ABD⁺25, BNRT26, Pol26, ADG⁺26] are not batched IBE schemes and hence not discussed in this work.

In this work we construct the first batched IBE scheme that supports q key queries *per label* with constant-size ciphertexts. Users still choose a label, but rather than guessing the single batch in which their transaction will be decrypted, they only guess a window of q batches. The CRS is reused across labels and the committee is never forced to rotate its public key. Concretely, q can be chosen large enough such that most valuable transactions in the mempool will be included with very high probability. Even if we are very conservative and choose $q = 2000$, the CRS size with BLS12-381 is ≈ 750 MB. In comparison to [FPTX25], our construction incurs a minimal overhead of one $\mathbb{G}_2$ element in the ciphertext and one $\mathbb{G}_1$ element in the partial decryption. But in exchange the committee is never forced to sample a fresh public key, and the CRS is orders of magnitude smaller.

Rather than just presenting our scheme (although the unrolled construction can be found in Appendix A), we take a step back and first study batched IBE through the lens of *multi-message signatures*. Here a single signing operation can sign an entire vector of messages, and a public open algorithm derives a succinct opening with which any individual message can be verified against the batch signature. We prove that this primitive is implied by every batched IBE scheme with a sufficiently large plaintext space, and conversely every multi-message signature with deterministic verification can be used to construct a batched IBE scheme given an extractable witness encryption scheme for its verification relation. Of course, in light of the duality between IBE and digital signatures identified by Naor [BF01] such a connection is not surprising. The real value lies in the insight it provides into the inner workings of batched IBE schemes, exposing the most important ideas.

Our contributions.

- *Unified definitions.* We give a single definition of batched IBE (Section 5.2) that captures all prior constructions in one framework, and an analogous definition of multi-message signatures (Section 5.1).
- *Batched IBE and Multi-Message Signatures.* We show that every batched IBE scheme with a sufficiently large plaintext space implies a multi-message signature scheme (Construction 2), and that every multi-message signature scheme with deterministic verification can be used to construct a batched IBE scheme given an extractable witness encryption scheme for the verification relation (Construction 1). Along the way, we formalize extractable witness encryption *in the presence of oracles* (Definition 11), which may be of independent interest, and observe that the compiler can be instantiated efficiently for *linearly-verifiable* multi-message signatures [BC16, BL20, GKPW24, GHK⁺25].
- *Survey of prior work.* We extract and prove secure, in the generic bilinear group model, the multi-message signatures at the heart of all prior batched IBE constructions [CGPW25, AFP25, FPTX25, GWWW25] (Section 7), tracing how constructions evolved, connecting them to existing signature schemes, and explaining why restrictions on signing/key queries are necessary.
- *A new construction.* We construct the first multi-message signature scheme that is labeled q -EUF-CMA secure (in the generic bilinear group model) with a constant-size verification key, constant-size signatures and openings, and a CRS of size $O(qB)$ (Section 7.4), yielding the first batched IBE scheme supporting q key queries per label with constant-size ciphertexts. Similar to [CGPW25, AFP25, FPTX25], our work also supports an FFT optimized $O(B \log^2 B)$ decryption algorithm [FK23] and non-interactive threshold key issuance.

2 Overview

Multi-message signatures. Naor [BF01] observed a connection between Identity-Based Encryption and Digital Signatures: that secret keys issued using the master secret key in IBE *are* in fact digital signatures on the identity. Our work starts by extending this duality to the batched setting: a batch key is a signature on a vector of messages, from which one can extract a verifiable signature on each individual message. We formalize the signature primitive as a *multi-message signature* (Section 5.1): a scheme in which Sign takes a

whole vector $\mathbf{m} = (m_1, \dots, m_B)$ and produces one short signature σ , a public algorithm Open derives from σ an opening op_j for each signed message, and $\text{Verify}(\text{crs}, \text{vk}, m, \sigma, \text{op}, [\text{tg}])$ checks a *single* message against the batch signature.

Unforgeability (Definition 6) demands that after adaptively many signing queries on batches of the adversary’s choice, no adversary can produce an accepting triple $(m^*, \sigma^*, \text{op}^*)$ for a message m^* that has not been queried before. Of course, to appropriately capture the caveats of different Batched IBE schemes, we restrict the set of admissible queries with labels, and per-label query bounds (Definition 4) together with stateful signers. The restriction on admissible queries gives rise to a hierarchy of security notions, one for each variant of the scheme. We say a scheme is *EUFCMA* secure if it withstands an unbounded number of signing queries, and *q-EUFCMA* secure if the adversary may make at most q signing queries in total. It is *labeled q-EUFCMA* secure if every query (and the forgery) carries a batch label tg and the adversary may make at most q queries *per label*, with no bound on the number of labels. In particular, we show that [CGPW25, AFP25, GWWW25] satisfy labeled 1-EUFCMA and [FPTX25] satisfies q -EUFCMA.

Batched IBE $\implies$ Multi-Message Signatures. We first show that every batched IBE scheme implies a multi-message signature (Construction 2 and Theorem 2). The construction is the analogue of Naor’s IBE-to-signature transform [BF01] where the signature on a batch $\mathbf{m}$ is the batch secret key for the identity vector $\mathbf{m}$. It can be verified by sampling a random plaintext μ , encrypting it to the identity m , and accepting if the candidate key successfully decrypts the ciphertext to recover μ . Unforgeability can be reduced to the adaptive security of the batched IBE scheme (Theorem 2). Depending on the size of the plaintext space, the verifier may need to repeat this test.

Multi-Message Signatures $\implies$ Batched IBE. Given a multi-message signature scheme with deterministic verification, we encrypt a message m to identity id by witness-encrypting m to the statement “there exists a signature–opening pair for id under $(\text{crs}, \text{vk}, [\text{tg}])$ ”:

$$\mathcal{R}_{\text{MMS}} = \{ ((\text{crs}, \text{vk}, \text{id}, [\text{tg}]), (\sigma, \text{op})) : \text{Sig.Verify}(\text{crs}, \text{vk}, \text{id}, \sigma, \text{op}, [\text{tg}]) = 1 \},$$

and the batch secret key is simply the signature on the identity vector (Construction 1). The size of the batched IBE key is the size of the signature on the identity vector, and the size of the ciphertext is the size of the witness encryption scheme’s ciphertext (which grows roughly with the size of the verification circuit). Thus, in order to have an *interesting* batched IBE scheme, signatures need to be sublinear in the batch size and verification must be independent of the batch size to have constant size ciphertexts.

Correctness is immediate, but security requires care. The natural idea for the proof is to use the distinguisher against the batched IBE scheme as a witness-encryption adversary, invoke extractability to obtain a witness (a signature–opening pair for the challenge identity), and present that pair as a forgery against the signature scheme. The issue is that the distinguisher requires access to a key-computation oracle in the batched IBE security game. But standard extractable witness encryption only quantifies over plain adversaries, so the extractor must be given the secret key in order to answer key queries from the distinguisher. This makes extraction vacuous as it could simply sign the challenge identity itself.

Following the O-SNARKs of Fiore and Nitulescu [FN16], we define extractable witness encryption *in the presence of an oracle family* (Definition 11) where the adversary is given access to the oracle, but the extractor is only given the *transcript* of oracle queries and answers. Instantiating the family with the MMS signing oracle (Definition 12), any distinguisher against the compiled scheme yields an extractor whose output is a valid signature and since the transcript only contains the adversary’s key queries, none of which contain the challenge identity, the extracted pair is an EUFCMA forgery (Theorem 1). Although witness encryption for general relations is a heavyweight assumption, [BC16, BL20, GKPW24, GHK⁺25] showed that for structured pairing based relations with linear verification where witness elements are not paired with each other, there exists an efficient witness encryption scheme. In Table 1 we present ciphertext sizes for the corresponding schemes.

Previous Approaches. Recall that to have an interesting and efficient batched IBE scheme, we need to ensure that in the multi-message signature scheme:

- signatures are sublinear in the batch size
- verification of openings is independent of the batch size
- witness elements (openings, and signatures) are not paired with each other

Suppose we dropped the last requirement. Then one can encode the messages $\mathbf{m}$ as the monic polynomial $f_{\mathbf{m}}(X) = \prod_{j \in [B]} (X - m_j)$ and commit to it using a KZG polynomial commitment [KZG10] $\text{com} = g^{f_{\mathbf{m}}(\tau)}$. This commitment can then be signed using any signature scheme and openings op_j simply show that the message of interest m_j is a root of the polynomial committed to in com . Because KZG has constant time verification, and signatures are constant sized we are done. The real challenge lies in satisfying the last requirement.

Now observe that in the above recipe, the commitment com , the signature σ , and the openings op_j are all witness elements and therefore cannot be paired with each other. They can only be paired with the verification key vk , the CRS crs , and the message m . As a consequence, the signatures on commitments are actually linearly homomorphic, which means that an adversary can produce signatures on linear combinations of commitments.² If we use a KZG as described above, then this poses a problem. Given signatures on two batches $\mathbf{m}_1, \mathbf{m}_2$, choosing $c_1 = f_{\mathbf{m}_2}(m^*)$ and $c_2 = -f_{\mathbf{m}_1}(m^*)$ produces a commitment

$$\text{com}^* = \text{com}_1^{c_1} \cdot \text{com}_2^{c_2} = g^{c_1 f_{\mathbf{m}_1}(\tau) + c_2 f_{\mathbf{m}_2}(\tau)}$$

that can be opened at a message m^* that was never signed. The most interesting aspect of a batched IBE scheme is how it prevents such an attack – it must simultaneously destroy the homomorphic property while preserving linear verification. Two main strategies have emerged so far which can be viewed as destroying the homomorphic property either in the signatures or in the commitments.

- *Labeled Signatures* [CGPW25, AFP25, GWWW25]: each signature is associated with a public batch label tg that can be paired with the witness elements. The homomorphic property of signatures is weakened to only admit linear combinations of signatures under the same label. By restricting signing to at most one query per label, we can prevent the above attack. The downside in the encrypted mempool application is that the encryptor must now *guess* the batch label in which their ciphertext will be decrypted.
- *Linearly Independent CRSs* [FPTX25]: instead of using the same CRS $(\{g^{\tau^i}\}_{i \in [B]}, h^\tau)$ for every commitment, sample q different CRSs $(\{g^{\kappa_s \tau^i}\}_{i \in [B], s \in [q]}, h^\tau)$, each with a different generator g^{κ_s} but sharing the same trapdoor τ . This creates linearly independent commitments for each query, and linear combinations of commitments can no longer be opened. Although users no longer have to guess the batch label, this limits the number of signing queries to q , after which the public key must be refreshed, forcing an expensive distributed key generation after every q decryptions.

Our Multi-Message Signature Scheme. As discussed in the introduction, neither of the above strategies is satisfactory for the encrypted mempool application. The former forces the encryptor to guess the batch label in which their ciphertext will be decrypted, and the latter forces an expensive distributed key generation after every q decryptions. Our solution is a best of both worlds approach that destroys the homomorphic property in both the signatures and the commitments. This allows us to support q signing queries per label tg without ever being forced to redo the distributed key generation. Users now guess a range of q batches

²In fact, this offers an explanation of why all batched IBE based schemes have epoch dependence issues whereas the *algebraic cancellation* based schemes [BFOQ25, BCF⁺25, ABD⁺25, BNRT26, Pol26, ADG⁺26] do not, although they can also be viewed as witness encryption schemes (see [Pol26, Section 6.1])

which is significantly easier than guessing the exact batch in which their ciphertext will be decrypted. We require an $O(qB)$ CRS but this can be reused across all labels.

We now revisit prior constructions [CGPW25, AFP25, FPTX25, GWWW25] and extract the multi-message signature at the heart of each to understand how they’ve evolved and describe how we adapt ideas to build the first batched IBE scheme supporting q key queries per label with constant size ciphertexts. We summarize the schemes in Table 1 and a formal proof of security for each of the multi-message signatures is provided in Section 7. In the verification equations below, we highlight the witness elements, i.e., the signature and openings that the decryptor computes in the Batched IBE construction.

Construction	Security	Ciphertext size	CRS size
[CGPW25, AFP25]	labeled 1-EUF-CMA	$3 \mathbb{G}_2$	$O(B)$
[GWWW25]	labeled 1-EUF-CMA	$3 \mathbb{G}_2$	$O(B)$
[FPTX25]	q -EUF-CMA	$2 \mathbb{G}_2$	$O(qB)$
This Work	labeled q -EUF-CMA	$3 \mathbb{G}_2$	$O(qB)$

Table 1: Comparison of the multi-message signatures with *linear* verification and ciphertext sizes (excluding message size for CPA security) when using the witness encryption compiler from [GKPW24, GHK⁺25]. Note that it’s always possible to switch groups for shorter $\mathbb{G}_1$ ciphertexts at the cost of a larger CRS and more expensive Open step.

CGPW25, AFP25 (Section 7.1) Recall that a BLS signature [BLS01] has the form:

$$vk = h^\alpha, \quad \sigma = H(\text{msg})^\alpha$$

which can be verified as:

$$e(\sigma, h) \stackrel{?}{=} e(H(\text{msg}), h^\alpha)$$

[CGPW25, AFP25] use a *shifted* BLS signature on the tuple (tg, com) defined as:

$$\sigma = (H(\text{tg}) \cdot \text{com}^{-1})^\alpha,$$

which can be verified as

$$e(\sigma, h) \cdot e(\text{com}, h^\alpha) \stackrel{?}{=} e(H(\text{tg}), h^\alpha).$$

The multi-message signature scheme on $\mathbf{m} = (m_1, \dots, m_B)$ can then be verified as:

$$\begin{aligned} e(\sigma, h) \cdot e(\text{com}, h^\alpha) &\stackrel{?}{=} e(H(\text{tg}), h^\alpha) \\ e(\pi_j, h^{\tau-m_j}) &\stackrel{?}{=} e(\text{com}, h) \end{aligned}$$

where $\pi_j = g^{q_j(\tau)}$ and $q_j(X) = f_{\mathbf{m}}(X)/(X - m_j)$.

Remark 1. Note that if we additionally provide $h^{\alpha\tau}$ as part of the verification key, then we can compress the two equations into:

$$e(\sigma, h) \cdot e(\pi_j, h^{\alpha(\tau-m_j)}) \stackrel{?}{=} e(H(\text{tg}), h^\alpha).$$

This was observed in [FPTX25, GWWW25] who have a single compressed verification equation for both the signature and the openings.

GWWW25 (Section 7.2) Recall that a Boneh–Boyen signature [BB04a] has the form:

$$\text{vk} = (e(g, h)^\alpha, h^\gamma, h^\delta), \quad \sigma = (\sigma_0 = g^\rho, \sigma_1 = g^{\alpha + \rho(\gamma + \delta \text{msg})})$$

for fresh randomness ρ , which can be verified as:

$$e(\sigma_1, h) \stackrel{?}{=} e(g, h)^\alpha \cdot e(\sigma_0, h^\gamma (h^\delta)^{\text{msg}})$$

[GWWW25] uses a *shifted* Boneh–Boyen signature on the tuple (tg, com) which extends the verification key to include h^β and signs a tuple (tg, com) as:

$$\sigma = (y, \sigma_0 = g^\rho, \sigma_1 = g^{\alpha + \rho(\gamma + \delta \text{tg})} \cdot \text{com}^{y\beta}),$$

for randomly sampled y . It can be verified as

$$e(\sigma_1, h) \stackrel{?}{=} e(g, h)^\alpha \cdot e(\sigma_0, h^\gamma (h^\delta)^{\text{tg}}) \cdot e(\text{com}^y, h^\beta).$$

The multi-message signature scheme on $\mathbf{m} = (m_1, \dots, m_B)$ can then be verified as:

$$\begin{aligned} e(\sigma_1, h) &\stackrel{?}{=} e(g, h)^\alpha \cdot e(\sigma_0, h^\gamma (h^\delta)^{\text{tg}}) \cdot e(\text{com}^y, h^\beta) \\ e(\pi_j, h^{\tau - m_j}) &\stackrel{?}{=} e(\text{com}^y, h) \end{aligned}$$

where $\pi_j = g^{yq_j(\tau)}$ and $q_j(X) = f_{\mathbf{m}}(X)/(X - m_j)$. As in Remark 1, providing $h^{\beta\tau}$ in the verification key compresses the two equations into one, eliminating the commitment altogether:

$$e(\sigma_1, h) \stackrel{?}{=} e(g, h)^\alpha \cdot e(\sigma_0, h^\gamma (h^\delta)^{\text{tg}}) \cdot e(\pi_j, h^{\beta(\tau - m_j)})$$

FPTX25 (Section 7.3) Abandons labels altogether and takes the second strategy: linearly independent commitments. The setup publishes

$$\left(g, g', \left\{ g^{\kappa_s \tau^k} \right\}_{s \in [q], 0 \leq k \leq B}, h, h^\tau \right)$$

where g' is an obviously sampled base with unknown discrete logarithm. The signing algorithm resembles the shifted BLS signature on the tuple $(\text{tg}, \text{com}_s)$ for $\text{com}_s = g^{\kappa_s f_{\mathbf{m}}(\tau)}$, with the hash value $H(\text{tg})$ replaced by the fixed base g' (one may think of g' as $H(0)$ for a canonical label) and the commitment entering with the opposite sign. The scheme never releases two signatures using the same κ_s , and as a result even though signatures are homomorphic and the adversary can produce signatures on linear combinations of commitments, it cannot successfully open a commitment to a message that was never signed. Thus the signature on $\mathbf{m} = (m_1, \dots, m_B)$ is of the form

$$\sigma = (s, (g' \cdot \text{com}_s)^\alpha), \quad \text{com}_s = g^{\kappa_s f_{\mathbf{m}}(\tau)},$$

which can be verified as:

$$\begin{aligned} e(\sigma_1, h) &\stackrel{?}{=} e(g', h^\alpha) \cdot e(\text{com}_s, h^\alpha) \\ e(\pi_j, h^{\tau - m_j}) &\stackrel{?}{=} e(\text{com}_s, h) \end{aligned}$$

where $\pi_j = g^{\kappa_s q_j(\tau)}$. Again, by giving out $h^{\alpha\tau}$ as part of the verification key, we can compress the two equations into one, eliminating the commitment:

$$e(\sigma_1, h) \stackrel{?}{=} e(g', h^\alpha) \cdot e(\pi_j, h^{\alpha(\tau - m_j)}).$$

At this point, a natural question to ask is what happens if we combine the two strategies: replace the fixed base g' with a per-label hash $H(\text{tg})$, and maintain a per-label slot counter, so that up to q signatures can be released under each label. Unfortunately, the two strategies do not compose securely. For any $q \geq 2$, there is a concrete attack to produce forgeries in the (labeled q -EUF-CMA) multi-message signature game:

- Query the *same* batch $\mathbf{m}$ under both labels in slot s_1 , receiving $\sigma = (H(\mathbf{tg}_1) \cdot \text{com})^\alpha$ and $\sigma' = (H(\mathbf{tg}_2) \cdot \text{com})^\alpha$ with the same commitment $\text{com} = g^{\kappa_{s_1} f_{\mathbf{m}}(\tau)}$. Compute

$$\Delta := \sigma \sigma'^{-1} = (H(\mathbf{tg}_1) H(\mathbf{tg}_2)^{-1})^\alpha$$

- Fix a fresh target m^* and query the batches $\mathbf{m}_i = (z_i, \bar{\mathbf{m}})$ for $z_1 \neq z_2$, so that $f_{\mathbf{m}_i}(X) = (X - z_i) \cdot f_{\bar{\mathbf{m}}}(X)$, under $(\mathbf{tg}_1, s_2)$ and $(\mathbf{tg}_2, s_2)$, receiving σ_1, σ_2 . Define

$$c_1 = \frac{z_2 - m^*}{z_2 - z_1}, \quad c_2 = \frac{m^* - z_1}{z_2 - z_1}, \quad \text{so that} \quad c_1 + c_2 = 1, \quad c_1(X - z_1) + c_2(X - z_2) = X - m^*,$$

and compute

$$\sigma^* := \sigma_1^{c_1} \sigma_2^{c_2} \cdot \Delta^{c_2} = \left(H(\mathbf{tg}_1) \cdot g^{\kappa_{s_2}(\tau - m^*) f_{\bar{\mathbf{m}}}(\tau)} \right)^\alpha,$$

which is a signature on the batch $(m^*, \bar{\mathbf{m}})$.

Our Construction. Our starting point (Section 7.4) is the shifted Boneh–Boyen signature on the tuple $(\mathbf{tg}, \text{com}_s)$ used in Section 7.2 but commitments are created using the *slotted* CRS used in Section 7.3. As in the threshold construction of [GWW25], the secrets (γ, δ) are placed in the trusted setup as $(g^\gamma, g^\delta, h^\gamma, h^\delta)$, which can be reused across different committees, and the random scalar y is dropped. We prove that security holds in the Generic Group Model even after setting $\alpha = \beta$. The signature on a tuple $(\mathbf{tg}, \text{com}_s)$ is then of the form

$$\text{vk} = h^\alpha, \quad \sigma = \left(s, \sigma_0 = g^\rho, \sigma_1 = (g \cdot \text{com}_s)^\alpha \cdot g^{\rho(\gamma + \delta \mathbf{tg})} \right), \quad \text{com}_s = g^{\kappa_s f_{\mathbf{m}}(\tau)},$$

and the corresponding multi-message signature scheme on $\mathbf{m} = (m_1, \dots, m_B)$ is verified as:

$$\begin{aligned} e(\sigma_1, h) &\stackrel{?}{=} e(g, h)^\alpha \cdot e(\text{com}_s, h^\alpha) \cdot e(\sigma_0, h^\gamma (h^\delta)^{\mathbf{tg}}) \\ e(\pi_j, h^{\tau - m_j}) &\stackrel{?}{=} e(\text{com}_s, h) \end{aligned}$$

where $\pi_j = g^{\kappa_s q_j(\tau)}$ and $q_j(X) = f_{\mathbf{m}}(X)/(X - m_j)$. As in Remark 1, additionally providing $h^{\alpha\tau}$ in the verification key compresses the two equations into one, eliminating the commitment altogether:

$$e(\sigma_1, h) \stackrel{?}{=} e(g, h)^\alpha \cdot e(\sigma_0, h^\gamma (h^\delta)^{\mathbf{tg}}) \cdot e(\pi_j, h^{\alpha(\tau - m_j)}).$$

The result is the first multi-message signature scheme that is labeled q -EUF-CMA secure with constant-size signatures and openings and a CRS of size $O(qB)$ (Table 1). Compiled through Construction 1 with the linear pairing-product witness encryption of [GKPW24], it yields the first labeled batched IBE scheme supporting q key queries per label with constant-size $(3\mathbb{G}_2)$ ciphertexts (Corollary 1). The unrolled batched IBE scheme can be found in Appendix A.

Our scheme can also support threshold signing/key issuance by Shamir secret sharing α and having the signers sample ρ independently. Partial signatures/key shares are $2\mathbb{G}_1$ elements, and signatures/keys can be recovered using standard Lagrange interpolation in the exponent.

3 Related Work

Homomorphic signatures. Homomorphic signatures [JMSW02] allow anyone to derive, from signatures on messages $m_1, \dots, m_n$, a signature on $f(m_1, \dots, m_n)$ for admissible functions f . Linearly homomorphic signature schemes developed in the context of network coding [BFKW09, GKRR10, AL11, Fre12] are the closest to our work. They sign vectors under a *file identifier* (analogous to a batch label), and signatures on linear combinations of vectors signed under the same file identifier can be publicly derived. In fact, the shifted BLS signature from [CGPW25, AFP25] has a very similar structure to that of [BFKW09], with the main

difference being the use of a KZG commitment, whose openings can be verified in constant time, instead of a Pedersen commitment, whose openings require verification time linear in the batch size. Similarly, the generic framework of [Fre12] builds linearly homomorphic signatures from Waters signatures [Wat05] and Boneh-Boyen signatures [BB04b, BB08] (although this is a different signature scheme than the one used in [GWWW25] and our construction). Later works extend the admissible functions to polynomials [BF11] and to arbitrary bounded-depth circuits [GVW15].

Anonymous Credentials. [CL03, BBS04, TZ23, PS16] are signature schemes that sign attribute vectors for the purpose of issuing anonymous credentials. However, the requirements in the credential setting are completely different where a zero-knowledge proof is used to demonstrate possession of a signature on a (subset) of messages. They impose no requirements on the succinctness of the proof (which is often allowed to grow linearly with the size of the message vector), and does not have to be linearly verifiable.

4 Preliminaries

We use $[n]$ to denote the set $\{1, 2, \dots, n\}$. The security parameter is denoted by $\lambda \in \mathbb{N}$. A function $f : \mathbb{N} \rightarrow \mathbb{N}$ is said to be polynomial if there exists a constant c such that $f(n) \leq n^c$ for all $n \in \mathbb{N}$, and we write $\text{poly}(\cdot)$ to denote such a function. A function $f : \mathbb{N} \rightarrow [0, 1]$ is said to be negligible if for every $c \in \mathbb{N}$, there exists $N \in \mathbb{N}$ such that for all $n > N$, $f(n) < n^{-c}$, and we write $\text{negl}(\cdot)$ to denote such a function. A probability is *noticeable* if it is not negligible, and *overwhelming* if it is equal to $1 - \text{negl}(\lambda)$ for some negligible function $\text{negl}(\lambda)$. For a set $\mathcal{S}$, we write $s \leftarrow \mathcal{S}$ to indicate that s is sampled uniformly at random from $\mathcal{S}$. An algorithm $\mathcal{A}$ is PPT (probabilistic polynomial-time) if its running time is bounded by some polynomial in the size of its input.

Bilinear groups. Throughout the paper, $\mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T$ denote cyclic groups of prime order p with generators $g \in \mathbb{G}_1$ and $h \in \mathbb{G}_2$, equipped with a non-degenerate, efficiently computable bilinear pairing $e : \mathbb{G}_1 \times \mathbb{G}_2 \rightarrow \mathbb{G}_T$. We write $\mathbb{F} = \mathbb{Z}_p$ for the field of exponents; all polynomials (batch polynomials, Shamir sharings) live in $\mathbb{F}[X]$.

Linearly verifiable relation. A relation $\mathcal{R}$ whose statements x and witnesses w consist of group elements and field elements is *linearly verifiable* if membership $(x, w) \in \mathcal{R}$ is decided by a fixed system of pairing-product equations in which every pairing takes at most one argument from w ; the remaining arguments and the $\mathbb{G}_T$ targets are computed from x alone. In particular, witness elements are never paired with each other.

Generic bilinear group model. We will use an extension of the generic group model [Sho97] adapted to bilinear groups. We follow the presentation in [BFKT24], in particular defining the label sets as known sets of size equal to the size of the group. Moreover, we define the Add interface to directly support arbitrary linear combinations of group elements, which is without loss of generality due to the double-and-add algorithm.

Definition 1 (Generic Bilinear Group Oracle). *A generic bilinear group oracle is a stateful oracle GGM, parameterized by a prime $p \in \mathbb{Z}$, and three sets S_1, S_2, S_T of size p .*

- On a query $\text{GGM.Init}()$, the oracle samples and stores three uniformly random injections $\iota_i : \mathbb{Z}_p \rightarrow S_i$ for $i \in \{1, 2, T\}$. Future queries to GGM.Init are ignored.
- On a query $\text{GGM.GetGenerator}(i)$, the oracle returns $\iota_i(1)$.
- On a query $\text{GGM.Add}(i, g_1, g_2, c_1, c_2)$, return $\perp$ unless $g_1, g_2 \in S_i$ and $c_1, c_2 \in \mathbb{Z}_p$. Otherwise, return $g = \iota_i(c_1 \iota_i^{-1}(g_1) + c_2 \iota_i^{-1}(g_2))$.
- On a query $\text{GGM.Pair}(g_1, g_2)$, return $\perp$ unless $g_1 \in S_1, g_2 \in S_2$. Otherwise, return $\iota_T(\iota_1^{-1}(g_1) \cdot \iota_2^{-1}(g_2))$.

Modeling a group-output random oracle. We will also model the scenario in which parties have access to a cryptographic hash function that outputs group elements, which is needed for example to support BLS signatures [BLS01]. Since the sets S_1, S_2, S_T are public, this is straightforward to implement via a random oracle defined from $\mathbb{Z}_p \rightarrow S_i$.

Definition 2. Let $\mathbf{L} = (\mathbf{L}_1, \mathbf{L}_2, \mathbf{L}_T)$ be three sets of polynomials with elements in $\mathbb{Z}_p[X_1, \dots, X_n]$. We define the completion of $\mathbf{L}$ to be $C(\mathbf{L}) = \{\mathbf{L}_1 \otimes \mathbf{L}_2\} \cup \mathbf{L}_T = \{f_i(X_1, \dots, X_n)\}_i$. A polynomial $f \in \mathbb{Z}_p[X_1, \dots, X_n]$ is said to be dependent on $\mathbf{L} = (\mathbf{L}_1, \mathbf{L}_2, \mathbf{L}_T)$ if there exist coefficients $\{c_i\}_i \in \mathbb{Z}_p$ such that:

$$f(X_1, \dots, X_n) = \sum_i c_i \cdot f_i(X_1, \dots, X_n).$$

Otherwise, we say that f is independent of $\mathbf{L}$.

Symbolic proofs. Our generic-group security proofs follow the standard symbolic template. The secrets sampled by the challenger (trapdoors, keys, randomizers) are treated as formal variables, so that every handle held by the adversary is associated with a formal polynomial: the exponents of the $\mathbb{G}_1$ handles it received in $\mathbf{L}_1$, those of the $\mathbb{G}_2$ handles in $\mathbf{L}_2$, and those of the $\mathbb{G}_T$ handles in $\mathbf{L}_T$. By the Schwartz–Zippel lemma [Sch80, Zip79], two handles whose formal polynomials are distinct collide with probability at most d/p per pair, where d bounds the total degree. A union bound over the polynomially many pairs shows that, except with negligible probability, the adversary learns nothing beyond the formal polynomials themselves. In particular, every $\mathbb{G}_1$ (resp. $\mathbb{G}_2$) exponent it can produce is dependent on (i.e. lies in the span of) the exponents of the $\mathbb{G}_1$ (resp. $\mathbb{G}_2$) handles it received, and every $\mathbb{G}_T$ exponent is dependent on the completion $C(\mathbf{L})$ of Definition 2. We refer to this negligible failure event as the *formal-polynomial collision probability*.

5 Definitions

In this section, we first present a unified definition of multi-message signatures (Section 5.1) across different variants: stateless/stateful, labeled/unlabeled, bounded/unbounded queries. We capture security guarantees through the notion of admissible query sequences (Definition 4). Next we present analogous definitions of batched identity-based encryption (Section 5.2) (stateless/stateful, labeled/unlabeled, bounded/unbounded queries), capturing all known variants of batched IBE [CGPW25, AFP25, FPTX25, GWWW25] and the new scheme that we introduce. Finally, we introduce the definition of extractable witness encryption in the presence of oracles (Section 5.3), which will be used to construct batched IBE from multi-message signatures. We note that all of the definitions can be extended to the threshold setting in a straightforward manner, but we focus on the single-signer to better highlight the most interesting aspects of the constructions.

5.1 Multi-Message Signatures

Definition 3. A multi-message signature scheme with batch size B is a tuple of algorithms

$$(\text{Setup}, \text{KeyGen}, \text{Sign}, \text{Open}, \text{Verify})$$

defined as follows. We write optional inputs in [brackets]: stateful schemes use signing state st , labeled schemes use a batch-label space $\mathcal{L}$, and bounded schemes use a query bound q , which is either an input to Setup or a constant fixed by the scheme (in which case it is omitted from the input). Stateless, unlabeled, or unbounded schemes omit the corresponding inputs.

- $\text{Setup}(1^\lambda, B, [q]) \rightarrow \text{crs}$: takes as input the security parameter λ , batch size B , and optionally a query bound q , and outputs common public setup material, which determines the message space $\mathcal{M}$ and, for labeled schemes, the batch-label space $\mathcal{L}$.

- $\text{KeyGen}(\text{crs}) \rightarrow (\text{vk}, \text{sk}, [\text{st}_0])$: outputs a verification key, a secret signing key, and optionally an initial signing state.
- $\text{Sign}(\text{crs}, \text{vk}, \text{sk}, \mathbf{m}, [\text{st}], [\text{tg}]) \rightarrow (\sigma, [\text{st}'])/\perp$: on input the common setup, verification key, secret key, message vector $\mathbf{m} = (m_1, \dots, m_B) \in \mathcal{M}^B$, optional signing state st , and optional batch label $\text{tg} \in \mathcal{L}$, outputs a batch signature σ and optionally an updated signing state st' . It may output $\perp$ if the state has exhausted the scheme's signing-query budget.
- $\text{Open}(\text{crs}, \text{vk}, \sigma, \mathbf{m}, [\text{tg}]) \rightarrow (\text{op}_1, \dots, \text{op}_B)/\perp$: on input the common setup, verification key, a batch signature, the signed vector, and optional batch label tg , outputs openings for each message, or $\perp$ on malformed inputs.
- $\text{Verify}(\text{crs}, \text{vk}, m, \sigma, \text{op}, [\text{tg}]) \rightarrow \{0, 1\}$: outputs 1 if op opens the batch signature σ to a signature on $m \in \mathcal{M}$ under the optional batch label tg , and outputs 0 otherwise.

Definition 4 (Admissible Query Sequences). A signing-query sequence for a multi-message signature scheme with batch size B and message space $\mathcal{M}$ is a sequence

$$(\mathbf{m}_1, [a_1]), \dots, (\mathbf{m}_\ell, [a_\ell])$$

with $\mathbf{m}_j = (m_{j,1}, \dots, m_{j,B}) \in \mathcal{M}^B$. The optional inputs are well formed if $a_j \in \mathcal{L}$ for every $j \in [\ell]$ in labeled schemes, and are omitted in unlabeled schemes. The sequence is admissible if the optional inputs are well formed and:

- for unbounded schemes, there is no restriction on ℓ ;
- for unlabeled query-bounded schemes, $\ell \leq q$;
- for labeled query-bounded schemes,

$$\forall a \in \mathcal{L} : |\{j \in [\ell] : a_j = a\}| \leq q.$$

Definition 5 (Perfect Correctness). Perfect correctness of a multi-message signature scheme with batch size B and message space $\mathcal{M}$ is defined with respect to an admissible signing-query sequence and demands that the signatures produced along that sequence always verify. Formally, for all

$$\text{crs} \leftarrow \text{Setup}(1^\lambda, B, [q]), \quad (\text{vk}, \text{sk}, [\text{st}_0]) \leftarrow \text{KeyGen}(\text{crs})$$

and for any admissible query sequence

$$(\mathbf{m}_1, [a_1]), \dots, (\mathbf{m}_\ell, [a_\ell])$$

in the sense of Definition 4, the signing algorithm must always succeed

$$\{(\sigma_j, [\text{st}_j]) \leftarrow \text{Sign}(\text{crs}, \text{vk}, \text{sk}, \mathbf{m}_j, [\text{st}_{j-1}], [a_j])\}_{j \in [\ell]},$$

and every signature must pass verification

$$\left\{ \begin{array}{l} (\text{op}_{j,1}, \dots, \text{op}_{j,B}) \leftarrow \text{Open}(\text{crs}, \text{vk}, \sigma_j, \mathbf{m}_j, [a_j]), \\ \text{Verify}(\text{crs}, \text{vk}, m_{j,k}, \sigma_j, \text{op}_{j,k}, [a_j]) = 1 \end{array} \right\}_{j \in [\ell], k \in [B]}$$

Definition 6 (EUF-CMA). Let $\mathcal{A}$ be a non-uniform PPT adversary. The EUF-CMA game for a multi-message signature scheme with batch size B , message space $\mathcal{M}$, and optional batch-label space $\mathcal{L}$ and query bound q is defined as follows:

1. The challenger runs $\text{crs} \leftarrow \text{Setup}(1^\lambda, B, [q])$ and $(\text{vk}, \text{sk}, [\text{st}]) \leftarrow \text{KeyGen}(\text{crs})$, and gives (crs, vk) to $\mathcal{A}$. It initializes an empty set Q_{msg} of messages that have been signed and an empty signing-query sequence.

2. $\mathcal{A}$ may adaptively query $(\mathbf{m}, [\text{tg}])$, where $\mathbf{m} = (m_1, \dots, m_B) \in \mathcal{M}^B$ and tg is present exactly for labeled schemes, so long as the resulting signing-query sequence is admissible in the sense of Definition 4. The challenger appends the query to the signing-query sequence and runs

$$y \leftarrow \text{Sign}(\text{crs}, \text{vk}, \text{sk}, \mathbf{m}, [\text{st}], [\text{tg}]).$$

If $y = \perp$, the challenger returns $\perp$ (for perfectly correct schemes this never happens on an admissible sequence). Otherwise it parses $y = (\sigma, [\text{st}'])$, updates $\text{st} \leftarrow \text{st}'$ if an updated state is returned, updates $Q_{\text{msg}} \leftarrow Q_{\text{msg}} \cup \{m_1, \dots, m_B\}$, and returns σ .

3. $\mathcal{A}$ outputs $(m^*, \sigma^*, \text{op}^*, [\text{tg}^*])$, where tg^* is only present in labeled schemes. It wins if $m^* \notin Q_{\text{msg}}$ and $\text{Verify}(\text{crs}, \text{vk}, m^*, \sigma^*, \text{op}^*, [\text{tg}^*]) = 1$.

The multi-message signature scheme is EUF-CMA secure, q -EUF-CMA secure, or labeled q -EUF-CMA secure, respectively, if for every polynomial $B = B(\lambda)$ and, for bounded schemes, every polynomial $q = q(\lambda)$ supported by the scheme, all non-uniform PPT adversaries win the game with negligible probability under the corresponding admissibility condition of Definition 4.

Schemes that take the query bound as a Setup input, such as FPTX25 and our construction (Section 7), support every polynomial q , whereas CGPW25/AFP25 and GWWW25 fix $q = 1$ and only admit one signing query per label.

Remark 2. In the EUF-CMA game Definition 6, it may be reasonable to strengthen the definition against adversaries that are allowed to query m^* under a different label $\text{tg} \neq \text{tg}^*$. However, the weaker security guarantee above is sufficient for the encrypted mempool application.

5.2 Batched Identity-Based Encryption

Definition 7 (Batched Identity-Based Encryption). A batched identity-based encryption scheme is a tuple of efficient algorithms

$$\Pi_{\text{BIBE}} = (\text{Setup}, \text{KeyGen}, \text{Enc}, \text{ComputeKey}, \text{Dec})$$

with the following syntax. We write optional inputs in brackets: stateful schemes use key-computation state st , labeled schemes use a batch-label space $\mathcal{T}$, and bounded schemes use a key-query bound q , which is either an input to Setup or a constant fixed by the scheme (in which case it is omitted from the input).

- $\text{Setup}(1^\lambda, B, [q]) \rightarrow \text{pp}$: on input the security parameter, batch size B , and optionally a key-query bound q , outputs public parameters pp . The public parameters determine a message space $\mathcal{M}$, identity space $\mathcal{I}$, and optionally a batch-label space $\mathcal{T}$.
- $\text{KeyGen}(\text{pp}) \rightarrow (\text{mpk}, \text{msk}, [\text{st}_0])$: on input the public parameters, outputs a master public key mpk , a master secret key msk , and optionally an initial key-computation state.
- $\text{Enc}(\text{mpk}, m, \text{id}, [\text{tg}]) \rightarrow \text{ct}$: on input a message $m \in \mathcal{M}$, an identity $\text{id} \in \mathcal{I}$, and optional batch label $\text{tg} \in \mathcal{T}$, outputs a ciphertext ct .
- $\text{ComputeKey}(\text{msk}, \text{id}, [\text{st}], [\text{tg}]) \rightarrow (\text{sk}, [\text{st}'])/\perp$: on input the master secret key, an identity vector $\text{id} = (\text{id}_1, \dots, \text{id}_B) \in \mathcal{I}^B$, optional key-computation state st , and optional batch label $\text{tg} \in \mathcal{T}$, outputs a secret key associated with the identity vector and label and optionally an updated state. The algorithm may output $\perp$ if the state has exhausted the scheme's key-query budget.
- $\text{Dec}(\text{mpk}, \text{sk}, \text{id}, \text{ct}, [\text{tg}]) \rightarrow m/\perp$: on input the master public key, a secret key, an identity vector, an identity, a ciphertext, and optional batch label $\text{tg} \in \mathcal{T}$, outputs a message $m \in \mathcal{M}$ or $\perp$.

Definition 8 (Admissible Key-Query Sequences). A key-computation query sequence for a batched IBE scheme with batch size B and identity space $\mathcal{I}$ is a sequence

$$(\mathbf{id}_1, [a_1]), \dots, (\mathbf{id}_\ell, [a_\ell])$$

with $\mathbf{id}_j = (\text{id}_{j,1}, \dots, \text{id}_{j,B}) \in \mathcal{I}^B$. The optional inputs are well formed if $a_j \in \mathcal{T}$ for every $j \in [\ell]$ in labeled schemes, and are omitted in unlabeled schemes. The sequence is admissible if the optional inputs are well formed and:

- for unbounded schemes, there is no restriction on ℓ ;
- for unlabeled query-bounded schemes, $\ell \leq q$;
- for labeled query-bounded schemes,

$$\forall a \in \mathcal{T} : |\{j \in [\ell] : a_j = a\}| \leq q.$$

Definition 9 (Perfect Correctness). Perfect correctness of a batched IBE scheme with batch size B is defined with respect to an admissible key-query sequence and demands that the secret keys produced along that sequence decrypt correctly. Formally, for all

$$\text{pp} \leftarrow \text{Setup}(1^\lambda, B, [q]), \quad (\text{mpk}, \text{msk}, [\text{st}_0]) \leftarrow \text{KeyGen}(\text{pp})$$

and for any admissible key-query sequence

$$(\mathbf{id}_1, [a_1]), \dots, (\mathbf{id}_\ell, [a_\ell])$$

in the sense of Definition 8, the key-computation algorithm must always succeed

$$\{(\text{sk}_j, [\text{st}_j]) \leftarrow \text{ComputeKey}(\text{msk}, \mathbf{id}_j, [\text{st}_{j-1}], [a_j])\}_{j \in [\ell]},$$

and every key must decrypt correctly: for all $j \in [\ell]$, all identities $\text{id} \in \{\text{id}_{j,1}, \dots, \text{id}_{j,B}\}$, and all messages $m \in \mathcal{M}$,

$$\Pr [\text{Dec}(\text{mpk}, \text{sk}_j, \mathbf{id}_j, \text{id}, \text{ct}, [a_j]) = m : \text{ct} \leftarrow \text{Enc}(\text{mpk}, m, \text{id}, [a_j])] = 1.$$

Definition 10 (Adaptive Security). Let $\mathcal{A}$ be a non-uniform PPT adversary. The adaptive security game for a batched IBE scheme with batch size B , optional batch-label space $\mathcal{T}$, and optional query bound q is defined as follows:

1. The challenger runs

$$\text{pp} \leftarrow \text{Setup}(1^\lambda, B, [q]), \quad (\text{mpk}, \text{msk}, [\text{st}]) \leftarrow \text{KeyGen}(\text{pp}),$$

gives $(1^\lambda, 1^B, \text{pp}, \text{mpk})$ to $\mathcal{A}$, and initializes an empty key-query sequence.

2. $\mathcal{A}$ may adaptively make key-computation queries $(\mathbf{id}, [\text{tg}])$, where $\mathbf{id} \in \mathcal{I}^B$ and tg is present exactly for labeled schemes, so long as the resulting key-query sequence is admissible in the sense of Definition 8. The challenger appends the query to the key-query sequence and runs

$$y \leftarrow \text{ComputeKey}(\text{msk}, \mathbf{id}, [\text{st}], [\text{tg}]).$$

If $y = \perp$, the challenger returns $\perp$ (for perfectly correct schemes this never happens on an admissible sequence). Otherwise it parses $y = (\text{sk}, [\text{st}'])$, updates $\text{st} \leftarrow \text{st}'$ if an updated state is returned, and returns sk .

3. $\mathcal{A}$ outputs two messages $m_0, m_1 \in \mathcal{M}$ and a challenge identity with optional label $(\text{id}^*, [\text{tg}^*])$, where tg^* is present exactly for labeled schemes. The challenger samples $\beta \xleftarrow{\$} \{0, 1\}$ and returns

$$\text{ct}^* \leftarrow \text{Enc}(\text{mpk}, m_\beta, \text{id}^*, [\text{tg}^*]).$$

4. $\mathcal{A}$ may continue to make key-computation queries, answered as above.
5. $\mathcal{A}$ outputs a bit β' .

The adversary is challenge-admissible if the key-query sequence is admissible and no key query $(\text{id}, [\text{tg}])$ contains id^* in the vector id . The scheme is adaptively secure, q -adaptively secure, or labeled q -adaptively secure, respectively, if for every polynomial $B = B(\lambda)$, for bounded schemes every polynomial $q = q(\lambda)$ supported by the scheme, and every challenge-admissible non-uniform PPT adversary $\mathcal{A}$,

$$\Pr[\beta' = \beta] \leq \frac{1}{2} + \text{negl}(\lambda),$$

under the corresponding admissibility condition of Definition 8.

5.3 Extractable Witness Encryption in the Presence of Oracles

Following the O -SNARKs of Fiore and Nitulescu [FN16], we define extractable witness encryption relative to an oracle family $\mathbb{O}$, a distribution over pairs $(\text{pub}, \mathcal{O})$ of a public string and a (possibly stateful) oracle: the adversary gets the oracle, while the extractor gets only the *transcript* of the adversary's queries and answers. In our instantiation (Definition 12), the MMS oracle family samples an MMS key pair and lets $\mathcal{O}$ answer signing queries under sk , while $\text{pub} = (\text{crs}, \text{vk})$ publishes the corresponding CRS and verification key.

Definition 11 (Extractable Witness Encryption in the Presence of Oracles). A witness encryption scheme for an NP relation R with plaintext space $\mathcal{M}$ consists of two efficient algorithms:

- $\text{WE.Enc}(1^\lambda, x, m) \rightarrow \text{ct}$: on input a statement x and plaintext $m \in \mathcal{M}$, outputs a ciphertext ct .
- $\text{WE.Dec}(w, \text{ct}) \rightarrow m/\perp$: on input a witness w and ciphertext ct , outputs a plaintext or $\perp$.

We require the following two properties:

- Perfect correctness. For every $(x, w) \in R$ and every plaintext $m \in \mathcal{M}$,

$$\Pr[\text{WE.Dec}(w, \text{ct}) = m : \text{ct} \leftarrow \text{WE.Enc}(1^\lambda, x, m)] = 1.$$

- Extractable security in the presence of the oracle family $\mathbb{O}$. For every non-uniform PPT adversary $\mathcal{A}$ and every polynomial Q , there exist a non-uniform PPT extractor $\mathcal{E}$ and a polynomial P such that for every $\lambda \in \mathbb{N}$ the following holds. Consider the game:

1. Sample $(\text{pub}, \mathcal{O}) \leftarrow \mathbb{O}$ and random coins r for $\mathcal{A}$, used across both of its stages.
2. Run $(x, m_0, m_1, \text{st}) \leftarrow \mathcal{A}^\mathcal{O}(1^\lambda, \text{pub}; r)$, where $m_0, m_1 \in \mathcal{M}$.
3. Sample $\beta \xleftarrow{\$} \{0, 1\}$ and compute $\text{ct} \leftarrow \text{WE.Enc}(1^\lambda, x, m_\beta)$.
4. Run $\beta' \leftarrow \mathcal{A}^\mathcal{O}(\text{st}, \text{ct})$.

Let qt denote the transcript of all oracle queries made by $\mathcal{A}$ in the game together with the oracle's answers. If

$$\Pr[\beta' = \beta] \geq \frac{1}{2} + \frac{1}{Q(\lambda)},$$

then

$$\Pr[(x, w) \in R : w \leftarrow \mathcal{E}(1^\lambda, \text{pub}, x, \text{ct}, m_0, m_1, \text{qt}, r)] \geq \frac{1}{P(\lambda)},$$

where both probabilities are over the game above.

Remark 3 (Idealized models). When the extraction game is run relative to an idealized model, such as the generic bilinear-group model of Definition 1, a single instance of the idealized oracles is shared by the oracle family, the challenger, the adversary, and the extractor. The transcript qt then additionally records the adversary's queries to the idealized oracles together with their answers, and the extractor $\mathcal{E}$ may make idealized-oracle queries of its own (cf. the treatment of oracles in O -SNARKs [FN16]).

6 Batched IBE $\leftrightarrow$ Multi-Message Signatures

In this section we first show how to construct a Batched IBE scheme from a Multi-Message Signature with deterministic verification given an extractable witness encryption for the verification relation, and then show that, conversely, every Batched IBE scheme with a sufficiently large plaintext space implies a Multi-Message Signature.

6.1 Batched IBE from Multi-Message Signatures

Construction 1 (Batched IBE from Multi-Message Signatures and Extractable Witness Encryption). *Let*

$$\Pi_{\text{MMS}} = (\text{Sig.Setup}, \text{Sig.KeyGen}, \text{Sig.Sign}, \text{Sig.Open}, \text{Sig.Verify})$$

be a multi-message signature scheme with deterministic verification whose message space contains the BIBE identity space $\mathcal{I}$. A BIBE key query is an identity vector $\mathbf{id} = (\text{id}_1, \dots, \text{id}_B) \in \mathcal{I}^B$, which is also a valid MMS message vector. In the labeled case, the BIBE label space is the MMS label space. Define the relation

$$\mathcal{R}_{\text{MMS}} = \{ ((\text{crs}, \text{vk}, \text{id}, [\text{tg}]), (\sigma, \text{op})) : \text{Sig.Verify}(\text{crs}, \text{vk}, \text{id}, \sigma, \text{op}, [\text{tg}]) = 1 \}.$$

Let $(\text{WE.Enc}, \text{WE.Dec})$ be a witness encryption scheme for $\mathcal{R}_{\text{MMS}}$ with plaintext space $\mathcal{M}$. The compiled BIBE scheme has the same plaintext space $\mathcal{M}$. Define

$$\Pi_{\text{BIBE}}[\Pi_{\text{MMS}}, \text{WE}] = (\text{Setup}, \text{KeyGen}, \text{Enc}, \text{ComputeKey}, \text{Dec})$$

as follows.

- Setup($1^\lambda, B, [q]$). Run $\text{crs} \leftarrow \text{Sig.Setup}(1^\lambda, B, [q])$ and output

$$\text{pp} = (\text{crs}, B, [q]).$$

- KeyGen(pp). Parse $\text{pp} = (\text{crs}, B, [q])$, run

$$(\text{vk}, \text{sk}_{\text{sig}}, [\text{st}_0]) \leftarrow \text{Sig.KeyGen}(\text{crs}),$$

and output

$$\text{mpk} = (\text{crs}, \text{vk}), \quad \text{msk} = (\text{crs}, \text{vk}, \text{sk}_{\text{sig}}), \quad [\text{st}_0].$$

- Enc(mpk, m , id , $[\text{tg}]$). Parse $\text{mpk} = (\text{crs}, \text{vk})$ and output

$$\text{ct} \leftarrow \text{WE.Enc}(1^\lambda, (\text{crs}, \text{vk}, \text{id}, [\text{tg}]), m).$$

- ComputeKey(msk, $\mathbf{id}$, $[\text{st}]$, $[\text{tg}]$). Parse $\text{msk} = (\text{crs}, \text{vk}, \text{sk}_{\text{sig}})$ and run

$$(\sigma, [\text{st}']) \leftarrow \text{Sig.Sign}(\text{crs}, \text{vk}, \text{sk}_{\text{sig}}, \mathbf{id}, [\text{st}], [\text{tg}]),$$

forwarding $\perp$ if Sig.Sign outputs $\perp$. Output $(\text{sk}, [\text{st}']) = (\sigma, [\text{st}'])$.

- Dec(mpk, sk, $\mathbf{id}$, id , ct , $[\text{tg}]$). Parse $\text{mpk} = (\text{crs}, \text{vk})$ and $\text{sk} = \sigma$. Parse $\mathbf{id} = (\text{id}_1, \dots, \text{id}_B)$ and output $\perp$ if $\text{id} \notin \{\text{id}_1, \dots, \text{id}_B\}$. Otherwise compute

$$(\text{op}_1, \dots, \text{op}_B) \leftarrow \text{Sig.Open}(\text{crs}, \text{vk}, \sigma, \mathbf{id}, [\text{tg}]).$$

Choose an index j such that $\text{id}_j = \text{id}$ and output

$$\text{WE.Dec}((\sigma, \text{op}_j), \text{ct}).$$

Remark 4. While Definition 3 does not impose any succinctness requirements on the multi-message signature, it is only interesting to build batched IBE if the signatures are succinct in the batch size, and the verification circuit is independent (or at least sublinear) in the batch size.

Definition 12 (MMS Signing-Oracle Family). For a multi-message signature scheme Π_{MMS} with batch size B and optional query bound q , the oracle family $\mathbb{O}[\Pi_{\text{MMS}}]$ samples

$$\text{crs} \leftarrow \text{Sig.Setup}(1^\lambda, B, [q]), \quad (\text{vk}, \text{sk}_{\text{sig}}, [\text{st}_0]) \leftarrow \text{Sig.KeyGen}(\text{crs}),$$

sets $\text{pub} = (\text{crs}, \text{vk})$, and lets $\mathcal{O}$ be the stateful oracle that maintains the signing-query sequence and, on a query $(\mathbf{m}, [\text{tg}])$, answers $\perp$ if appending the query would render the sequence inadmissible in the sense of Definition 4. Otherwise it appends the query, responds with the signature output by $\text{Sig.Sign}(\text{crs}, \text{vk}, \text{sk}_{\text{sig}}, \mathbf{m}, [\text{st}], [\text{tg}])$, and updates its state. All other queries are answered with $\perp$.

Theorem 1. In Construction 1, suppose Π_{MMS} is correct and EUF-CMA secure under a certain admissibility condition, and WE is a secure witness encryption scheme for $\mathcal{R}_{\text{MMS}}$ in the presence of the oracle family $\mathbb{O}[\Pi_{\text{MMS}}]$ (Definitions 11 and 12). Then $\Pi_{\text{BIBE}}[\Pi_{\text{MMS}}, \text{WE}]$ is correct and adaptively secure under the corresponding BIBE admissibility condition.

Proof. Correctness follows directly from correctness of the multi-message signature scheme and witness encryption. Let $\text{id} = (\text{id}_1, \dots, \text{id}_B) \in \mathcal{I}^B$ and let $\text{id} \in \{\text{id}_1, \dots, \text{id}_B\}$. The key-computation algorithm produces a signature σ on id . Choose j such that $\text{id}_j = \text{id}$. By correctness of the signature scheme,

$$\text{Sig.Verify}(\text{crs}, \text{vk}, \text{id}, \sigma, \text{op}_j, [\text{tg}]) = 1$$

for the opening op_j output by Sig.Open . Thus (σ, op_j) is a valid witness for the statement $(\text{crs}, \text{vk}, \text{id}, [\text{tg}]) \in \mathcal{R}_{\text{MMS}}$, and correctness of witness encryption gives decryption of the encrypted plaintext.

We next prove adaptive security. Suppose for contradiction there is a challenge-admissible PPT adversary $\mathcal{A}$ that wins the adaptive security game for $\Pi_{\text{BIBE}}[\Pi_{\text{MMS}}, \text{WE}]$ with probability $\frac{1}{2} + \epsilon$ for non-negligible ϵ . We can then construct an adversary $\mathcal{A}'$ in the extraction game of Definition 11 for the oracle family $\mathbb{O}[\Pi_{\text{MMS}}]$. On input $(1^\lambda, \text{pub} = (\text{crs}, \text{vk}))$, the adversary $\mathcal{A}'$ sets $\text{pp} = (\text{crs}, B, [q])$ and $\text{mpk} = (\text{crs}, \text{vk})$, runs $\mathcal{A}$ on $(1^\lambda, 1^B, \text{pp}, \text{mpk})$, and answers each key-computation query $(\text{id}, [\text{tg}])$ by forwarding it to its oracle $\mathcal{O}$ and returning the resulting signature (or $\perp$) as the BIBE secret key. When $\mathcal{A}$ outputs its challenge $(m_0, m_1, \text{id}^*, [\text{tg}^*])$, adversary $\mathcal{A}'$ outputs the statement $x = (\text{crs}, \text{vk}, \text{id}^*, [\text{tg}^*])$ together with (m_0, m_1) . On receiving the challenge ciphertext ct , it forwards ct to $\mathcal{A}$, continues to answer key-computation queries through $\mathcal{O}$, and finally outputs $\mathcal{A}$'s bit β' .

The oracle family samples $(\text{crs}, \text{vk}, \text{sk}_{\text{sig}})$ exactly as the BIBE game, and challenge admissibility of $\mathcal{A}$ ensures every forwarded query is admissible, so $\mathcal{A}'$ simulates the BIBE game for $\mathcal{A}$ perfectly. Thus we have,

$$\Pr[\beta' = \beta] \geq \frac{1}{2} + \epsilon,$$

which is at least $\frac{1}{2} + \frac{1}{Q(\lambda)}$ for some polynomial Q . Extractable security in the presence of $\mathbb{O}[\Pi_{\text{MMS}}]$ then gives a PPT extractor $\mathcal{E}$ for $\mathcal{A}'$ that succeeds with probability at least $1/P(\lambda)$ for some polynomial P .

We now construct a PPT adversary $\mathcal{B}$ against EUF-CMA security of Π_{MMS} . The adversary $\mathcal{B}$ receives (crs, vk) from its challenger and has access to the MMS signing oracle. It runs $\mathcal{A}'$ on $(1^\lambda, \text{pub} = (\text{crs}, \text{vk}))$ with fresh coins r , relaying every oracle query of $\mathcal{A}'$ to its own signing oracle and recording the transcript qt of queries and answers. When $\mathcal{A}'$ outputs (x, m_0, m_1) with $x = (\text{crs}, \text{vk}, \text{id}^*, [\text{tg}^*])$, adversary $\mathcal{B}$ samples $\beta \xleftarrow{\$} \{0, 1\}$, computes

$$\text{ct} \leftarrow \text{WE.Enc}(1^\lambda, x, m_\beta),$$

resumes $\mathcal{A}'$ on ct , and continues relaying oracle queries. When $\mathcal{A}'$ halts, $\mathcal{B}$ runs

$$(\sigma^*, \text{op}^*) \leftarrow \mathcal{E}(1^\lambda, \text{pub}, x, \text{ct}, m_0, m_1, \text{qt}, r)$$

and outputs the forgery $(id^*, \sigma^*, op^*, [tg^*])$.

The EUF-CMA challenger samples (crs, vk, sk_{sig}) and answers signing queries exactly as $\mathcal{O}[\Pi_{MMS}]$, and $\mathcal{B}$ samples β and ct exactly as the extraction game, so $\mathcal{B}$ runs $\mathcal{E}$ on inputs distributed identically to that game. Hence, with probability at least $1/P(\lambda)$, the extracted pair satisfies $(x, (\sigma^*, op^*)) \in \mathcal{R}_{MMS}$, that is,

$$\text{Sig.Verify}(crs, vk, id^*, \sigma^*, op^*, [tg^*]) = 1.$$

Finally, the extractor is given only the transcript qt and makes no oracle queries of its own, so the messages signed in $\mathcal{B}$'s game are exactly the entries of $\mathcal{A}$'s key-computation queries, none of which contains id^* by challenge admissibility. Therefore $id^* \notin Q_{msg}$ and $(id^*, \sigma^*, op^*, [tg^*])$ is a valid EUF-CMA forgery, output with non-negligible probability. This contradicts the assumed unforgeability of Π_{MMS} . $\square$

6.2 Batched IBE implies Multi-Message Signatures

Construction 2 (Multi-Message Signatures from Batched IBE). *Let*

$$\Pi_{BIBE} = (\text{BIBE.Setup}, \text{BIBE.KeyGen}, \text{BIBE.Enc}, \text{BIBE.ComputeKey}, \text{BIBE.Dec})$$

be a batched IBE scheme with identity space $\mathcal{I}$ and plaintext space $\mathcal{M}_{BIBE}$ of super-polynomial size $|\mathcal{M}_{BIBE}| = \lambda^{\omega(1)}$ that supports efficient uniform sampling. Define a multi-message signature scheme $\Pi_{MMS}[\Pi_{BIBE}]$ with message space $\mathcal{I}$ as follows.

- MMS.Setup $(1^\lambda, B, [q])$. *Run*

$$pp \leftarrow \text{BIBE.Setup}(1^\lambda, B, [q])$$

and output $crs = pp$.

- MMS.KeyGen (crs) . *Parse* $crs = pp$ *and run*

$$(mpk, msk, [st_0]) \leftarrow \text{BIBE.KeyGen}(pp)$$

and output

$$vk = mpk, \quad sk = msk, \quad [st_0].$$

- MMS.Sign $(crs, vk, sk, \mathbf{m}, [st], [tg])$. *Parse* $vk = mpk$ *and* $sk = msk$, *and run*

$$(sk_{\mathbf{m}}, [st']) \leftarrow \text{BIBE.ComputeKey}(msk, \mathbf{m}, [st], [tg]),$$

forwarding $\perp$ *if* BIBE.ComputeKey *outputs* $\perp$. *Output* $(\sigma, [st']) = (sk_{\mathbf{m}}, [st'])$.

- MMS.Open $(crs, vk, \sigma, \mathbf{m}, [tg])$. *Output* $op_j = \mathbf{m}$ *for every* $j \in [B]$.
- MMS.Verify $(crs, vk, m, \sigma, op, [tg])$. *Parse* $vk = mpk$ *and* $op = \mathbf{m} = (m_1, \dots, m_B)$. *If* $m \notin \{m_1, \dots, m_B\}$, *output* 0. *Otherwise sample* $\mu \xleftarrow{\$} \mathcal{M}_{BIBE}$, *compute*

$$ct \leftarrow \text{BIBE.Enc}(mpk, \mu, m, [tg]),$$

and output 1 *iff*

$$\text{BIBE.Dec}(mpk, \sigma, \mathbf{m}, m, ct, [tg]) = \mu.$$

Theorem 2. *If Π_{BIBE} is correct and adaptively secure under a certain admissibility condition, then the scheme in Construction 2 is correct and EUF-CMA secure under the corresponding MMS admissibility condition.*

Proof. Correctness follows from BIBE correctness. For any honestly generated signature $\sigma = \text{sk}_{\mathbf{m}}$ on a vector $\mathbf{m}$ and any $m = m_j$, the verification algorithm samples μ , encrypts μ to m , and accepts because

$$\text{BIBE.Dec}(\text{mpk}, \text{sk}_{\mathbf{m}}, \mathbf{m}, m, \text{ct}, [\text{tg}]) = \mu.$$

For security, suppose there is a PPT adversary $\mathcal{A}$ that *wins* the EUF-CMA game for the derived signature scheme with non-negligible probability ϵ i.e. it produces a forgery on $m^* \notin Q_{\text{msg}}$. We build a challenge-admissible BIBE adversary $\mathcal{B}$ using $\mathcal{A}$. The BIBE challenger gives $\mathcal{B}$ the public parameters and master public key (pp, mpk) who then gives $\text{crs} = \text{pp}$ and $\text{vk} = \text{mpk}$ to $\mathcal{A}$, and initializes an empty set Q_{msg} .

To answer a signing query $(\mathbf{m}, [\text{tg}])$, adversary $\mathcal{B}$ makes the BIBE key query $(\mathbf{m}, [\text{tg}])$, returns the resulting BIBE secret key as the signature, and adds the entries of $\mathbf{m}$ to Q_{msg} , mirroring the EUF-CMA challenger. The key-query sequence is admissible whenever $\mathcal{A}$'s signing-query sequence is admissible.

Eventually $\mathcal{A}$ halts with an output. If the output fails to parse as a forgery $(m^*, \sigma^*, \text{op}^*, [\text{tg}^*])$ with $\text{op}^* = \mathbf{m}^*$ a vector containing m^* , or if $m^* \in Q_{\text{msg}}$, then $\mathcal{A}$ has certainly failed, and $\mathcal{B}$ simply outputs a uniform bit (β is independent of $\mathcal{B}$'s view here, so these executions are correct with probability exactly $\frac{1}{2}$). Otherwise $\mathcal{B}$ samples two *distinct* uniform plaintexts $\mu_0 \neq \mu_1$ from $\mathcal{M}_{\text{BIBE}}$ and submits them as challenge messages with identity $(m^*, [\text{tg}^*])$ to the BIBE challenger and receives

$$\text{ct}^* \leftarrow \text{BIBE.Enc}(\text{mpk}, \mu_\beta, m^*, [\text{tg}^*]).$$

It computes

$$\mu' \leftarrow \text{BIBE.Dec}(\text{mpk}, \sigma^*, \mathbf{m}^*, m^*, \text{ct}^*, [\text{tg}^*])$$

and outputs 0 if $\mu' = \mu_0$, outputs 1 if $\mu' = \mu_1$, and a uniform bit otherwise.

Since μ_β is marginally uniform, the pair (μ_β, ct^*) is distributed exactly as in the verification test of Construction 2, whose remaining checks $\mathcal{B}$ already enforced before challenging. Thus $\mathcal{A}$ wins the simulated game exactly when $\mathcal{B}$ reaches the challenge phase and $\mu' = \mu_\beta$, where $\mathcal{B}$ correctly outputs β which happens with probability ϵ . Conversely, $\mathcal{B}$ outputs the wrong bit with certainty only when $\mu' = \mu_{1-\beta}$, which happens with probability at most $\frac{1}{|\mathcal{M}_{\text{BIBE}}|-1}$ since $\mu_{1-\beta}$ is independent of μ' and everywhere else its guess is uniform. Splitting over these three disjoint cases,

$$\Pr[\beta' = \beta] = \epsilon + \frac{1}{2} (1 - \epsilon - \Pr[\mu' = \mu_{1-\beta}]) \geq \epsilon + \frac{1}{2} \left(1 - \epsilon - \frac{1}{|\mathcal{M}_{\text{BIBE}}| - 1} \right) = \frac{1}{2} + \frac{\epsilon}{2} - \frac{1}{2(|\mathcal{M}_{\text{BIBE}}| - 1)}.$$

Since $|\mathcal{M}_{\text{BIBE}}|$ is super-polynomial and ϵ is non-negligible, $\mathcal{B}$ is a challenge-admissible PPT adversary that wins the adaptive security game with probability noticeably above $\frac{1}{2}$, contradicting BIBE adaptive security. $\square$

7 Constructions

In this section, we extract multi-message signature schemes from previously known batched IBE schemes [CGPW25, AFP25, FPTX25, GWWW25], and finally present our construction (Section 7.4) which is the first multi-message signature scheme that satisfies labeled q -EUF-CMA security with a constant-size verification key, constant-size signatures and openings, and CRS size $O(qB)$. In all constructions, we denote

$$f_{\mathbf{m}}(X) = \prod_{j \in [B]} (X - m_j).$$

the monic degree- B polynomial whose multiset of roots is the batch $\mathbf{m}$.

In CGPW25/AFP25 (Section 7.1), FPTX25 (Section 7.3), and our construction (Section 7.4), the openings $\{\pi_j\}_{j \in [B]}$ are KZG openings of $f_{\mathbf{m}}$ (under the appropriate CRS slot), so all B openings can be computed together in $O(B \log^2 B)$ group operations using the standard technique of Feist and Khovratovich [FK23], as done in prior work [CGPW25, AFP25, FPTX25].

7.1 CGPW25, AFP25

Construction 3. Let $H : \{0, 1\}^* \rightarrow \mathbb{G}_1$ be a random oracle.

- Setup($1^\lambda, B$). Sample $\tau \xleftarrow{\$} \mathbb{F}$ and output

$$\text{crs} = \left(g, \{g^{\tau^k}\}_{0 \leq k \leq B}, h, h^\tau \right).$$

- KeyGen(crs). Sample $\alpha \xleftarrow{\$} \mathbb{F}$ and output

$$\text{vk} = \text{pk} = h^\alpha, \quad \text{sk} = \alpha, \quad \text{st}_0 = \emptyset,$$

where the state records the set of labels already used for signing.

- Sign(crs, vk, sk, **m**, st, tg). Parse $\text{sk} = \alpha$. If $\text{tg} \in \text{st}$, output $\perp$. Otherwise compute $\text{com} = g^{f_{\mathbf{m}}(\tau)}$ from crs and output

$$\sigma = (H(\text{tg}) \cdot \text{com}^{-1})^\alpha, \quad \text{st}' = \text{st} \cup \{\text{tg}\}.$$

- Open(crs, vk, σ , **m**). For each $j \in [B]$, compute $\text{com} = g^{f_{\mathbf{m}}(\tau)}$ and $q_j(X) = f_{\mathbf{m}}(X)/(X - m_j)$, and output $\text{op}_j = (\text{com}, \pi_j = g^{q_j(\tau)})$.
- Verify(crs, vk, m , σ , op, tg). Parse $\text{op} = (\text{com}, \pi)$ and accept iff

$$e(\sigma, h) \cdot e(\text{com}, \text{pk}) \stackrel{?}{=} e(H(\text{tg}), \text{pk}),$$

$$e(\pi, h^{\tau-m}) \stackrel{?}{=} e(\text{com}, h).$$

Theorem 3. In the generic bilinear-group model of Definition 1, with random oracle H , Construction 3 is a correct and labeled 1-EUF-CMA secure multi-message signature scheme with $\mathcal{L} = \{0, 1\}^*$ and batch size B .

Proof. Correctness. Consider any honest signing query on a fresh batch label tg and message vector **m** = $(m_1, \dots, m_B)$. Since tg has not been used before, the signing algorithm does not output $\perp$. It computes

$$\text{com} = g^{f_{\mathbf{m}}(\tau)} \quad \text{and} \quad \sigma = (H(\text{tg}) \cdot \text{com}^{-1})^\alpha.$$

Therefore, using $\text{pk} = h^\alpha$ and bilinearity,

$$e(\sigma, h) = e((H(\text{tg}) \cdot \text{com}^{-1})^\alpha, h) = e(H(\text{tg}) \cdot \text{com}^{-1}, h^\alpha) = e(H(\text{tg}) \cdot \text{com}^{-1}, \text{pk}).$$

For each $j \in [B]$, the opening algorithm outputs $\text{op}_j = (\text{com}, \pi_j = g^{q_j(\tau)})$, where $q_j(X) = f_{\mathbf{m}}(X)/(X - m_j)$. Hence

$$q_j(\tau)(\tau - m_j) = f_{\mathbf{m}}(\tau),$$

and so

$$e(\pi_j, h^{\tau-m_j}) = e(g^{q_j(\tau)}, h^{\tau-m_j}) = e(g^{f_{\mathbf{m}}(\tau)}, h) = e(\text{com}, h).$$

Thus every honestly generated signature opens and verifies for all $j \in [B]$.

Labeled 1-EUF-CMA security. Let A, T be formal variables for the secret key α and the trapdoor τ . The random oracle is implemented lazily: the first time H is evaluated on a label tg it is assigned a fresh independent formal variable Z_{tg} , so that $H(\text{tg}) = g^{Z_{\text{tg}}}$. Every formal polynomial in the game has total degree at most $B + 2$ in the formal variables, so the formal-polynomial collision probability is negligible. Up to this collision probability, every $\mathbb{G}_1$ exponent produced in the game lies in the span of

$$1, T, \dots, T^B, \quad \{Z_{\text{tg}}\}_{\text{tg}}, \quad \{A(Z_{\text{tg}_i} - f_{\mathbf{m}_i}(T))\}_i,$$

where tg ranges over all labels on which H has been evaluated and i over the successful signing queries. Thus the only source of an AZ_{tg} monomial is the unique signature returned for label tg .

Suppose $\mathcal{A}$ outputs an accepting forgery $(\text{tg}^*, m^*, \sigma^*, \text{op}^*)$ with $\text{op}^* = (\text{com}^*, \pi^*)$. Write the formal exponents of $\sigma^*, \text{com}^*, \pi^*$ as S, C, Π , respectively. The verification equations enforce that

$$C = (T - m^*)\Pi$$

and

$$S = A(Z_{\text{tg}^*} - C).$$

Since C and Π are $\mathbb{G}_1$ exponents, they lie in the span displayed above:

$$\begin{aligned} C &= \sum_{k=0}^B a_k T^k + \sum_{\text{tg}} b_{\text{tg}} Z_{\text{tg}} + \sum_i c_i A(Z_{\text{tg}_i} - f_{\mathbf{m}_i}(T)), \\ \Pi &= \sum_{k=0}^B a'_k T^k + \sum_{\text{tg}} b'_{\text{tg}} Z_{\text{tg}} + \sum_i c'_i A(Z_{\text{tg}_i} - f_{\mathbf{m}_i}(T)). \end{aligned}$$

The right-hand side of the opening equation $C = (T - m^*)\Pi$ contains the monomials TZ_{tg} and ATZ_{tg_i} with coefficients b'_{tg} and c'_i while the representation of C contains neither. Hence $b'_{\text{tg}} = c'_i = 0$ for all tg and i , and Π is a polynomial in T alone. Comparing the Z_{tg} and AZ_{tg_i} coefficients of $C = (T - m^*)\Pi(T)$ then gives $b_{\text{tg}} = 0$ and $c_i = 0$ as well. Thus C can be written as a polynomial in T alone, of degree at most B :

$$C = F(T),$$

$F(T)$ is divisible by $T - m^*$, and $\Pi = F(T)/(T - m^*)$ is a polynomial in T alone of degree at most $B - 1$. Since $(\text{tg}^*, m^*, \sigma^*, \text{op}^*)$ is an accepting forgery, $m^* \notin Q_{\text{msg}}$. One of two cases must hold:

1. No signing query used tg^* . In the span representation of S , the monomial AZ_{tg^*} can only appear from a returned signature for label tg^* so its coefficient in S is zero. But the signature equation

$$S = A(Z_{\text{tg}^*} - F(T))$$

contains this monomial with coefficient one, leading to a contradiction.

2. Some signing query used tg^* . Note that there is exactly one query (say $\mathbf{m}_i$) per label, where the signature has the form

$$\sigma_i = g^{A(Z_{\text{tg}^*} - f_{\mathbf{m}_i}(T))}.$$

Writing $S = \sum_k \tilde{a}_k T^k + \sum_{\text{tg}} \tilde{b}_{\text{tg}} Z_{\text{tg}} + \sum_j \tilde{c}_j A(Z_{\text{tg}_j} - f_{\mathbf{m}_j}(T))$ and comparing against $S = A(Z_{\text{tg}^*} - F(T))$ we see that every monomial on the right contains A , so $\tilde{a}_k = \tilde{b}_{\text{tg}} = 0$. For each j with $\text{tg}_j \neq \text{tg}^*$, the monomial AZ_{tg_j} appears on the left with coefficient $\tilde{c}_j$ and not at all on the right, so $\tilde{c}_j = 0$. Comparing AZ_{tg^*} gives $\tilde{c}_i = 1$, and the remaining terms give $F(T) = f_{\mathbf{m}_i}(T)$. Since $m^* \notin Q_{\text{msg}}$, the polynomial $f_{\mathbf{m}_i}(T)$ is not divisible by $T - m^*$, contradicting the divisibility of $F(T)$. □

Remark 5. The threshold construction in the appendix of [GWWW25] (their Construction E.2) is similar to the scheme above: signatures are deterministic and of the form $H(\text{tg})^\alpha \cdot \text{com}^w$, for two independently sampled secrets α and w . Both are proven secure in the generic group model.

7.2 GWWW25

Construction 4. • Setup($1^\lambda, B$). Sample $\tau \xleftarrow{\$} \mathbb{F}$ and output

$$\text{crs} = \left(g, \{g^{\tau^k}\}_{0 \leq k \leq B}, h, h^\tau \right).$$

• KeyGen(crs). Sample $\alpha, \beta, \gamma, \delta \xleftarrow{\$} \mathbb{F}$. Set $\text{pk} = h^\beta$ and $\text{pk}^\tau = h^{\beta\tau}$. Output

$$\text{vk} = (\text{pk}, \text{pk}^\tau, e(g, h)^\alpha, h^\gamma, h^\delta), \quad \text{sk} = (\alpha, \beta, \gamma, \delta), \quad \text{st}_0 = T.$$

where $T = \emptyset$ records the labels already used for signing.

• Sign(crs, vk, sk, **m**, st, **tg**). Parse $\text{sk} = (\alpha, \beta, \gamma, \delta)$ and $\text{st} = T$. If $\text{tg} \in T$, output $\perp$. Otherwise sample $\rho \xleftarrow{\$} \mathbb{F}$ and $y \xleftarrow{\$} \mathbb{F}^*$, set $T \leftarrow T \cup \{\text{tg}\}$, and output the updated state and the succinct signature

$$\left(\sigma = (y, \sigma_0 = g^\rho, \sigma_1 = g^{\alpha + y\beta f_{\mathbf{m}}(\tau) + \rho(\gamma + \delta \text{tg})}), \text{st}' = T \right).$$

• Open(crs, vk, σ , **m**). Parse $\sigma = (y, \sigma_0, \sigma_1)$. For each $j \in [B]$, compute $q_j(X) = f_{\mathbf{m}}(X)/(X - m_j)$ and output

$$\pi_j = g^{yq_j(\tau)}.$$

• Verify(crs, vk, m , σ , π , **tg**). Parse $\sigma = (y, \sigma_0, \sigma_1)$. Accept iff

$$e(\sigma_1, h) \stackrel{?}{=} e(g, h)^\alpha \cdot e(\sigma_0, h^\gamma (h^\delta)^{\text{tg}}) \cdot e(\pi, \text{pk}^{\tau - m}).$$

Theorem 4. In the generic bilinear-group model of Definition 1, Construction 4 is correct and labeled 1-EUF-CMA secure.

Proof. Correctness. On a signing query (**tg**, **m**), the signer outputs

$$\sigma_1 = g^{\alpha + y\beta f_{\mathbf{m}}(\tau) + \rho(\gamma + \delta \text{tg})}.$$

For an opening at m_j , write $q_j(X) = f_{\mathbf{m}}(X)/(X - m_j)$, so $\pi_j = g^{yq_j(\tau)}$. Then

$$e(g, h)^\alpha \cdot e(\sigma_0, h^\gamma (h^\delta)^{\text{tg}}) \cdot e(\pi_j, \text{pk}^{\tau - m_j}) = e(g, h)^{\alpha + \rho(\gamma + \delta \text{tg}) + y\beta q_j(\tau)(\tau - m_j)} = e(\sigma_1, h).$$

Labeled 1-EUF-CMA security. Let A, T, U, V, W be formal variables for $\alpha, \tau, \beta, \gamma, \delta$. We also set R_i to be the formal variable corresponding to the randomization ρ_i sampled when signing query (**tg**_{*i*}, **m**_{*i*}). Every $\mathbb{G}_1$ exponent available to the adversary lies in the span of

$$1, T, \dots, T^B, \quad \{R_i\}_i, \quad \{A + y_i U f_{\mathbf{m}_i}(T) + R_i(V + W \text{tg}_i)\}_i,$$

except with the usual formal-polynomial collision probability.

Suppose an accepting forgery is (**tg**^{*}, **m**^{*}, σ^* , π^*) with $\sigma^* = (y^*, \sigma_0^*, \sigma_1^*)$. Write the formal exponents of $\sigma_0^*, \sigma_1^*, \pi^*$ as R, S, Π . Verification gives

$$S = A + R(V + W \text{tg}^*) + U(T - m^*)\Pi. \tag{1}$$

Write $P_i := A + y_i U f_{\mathbf{m}_i}(T) + R_i(V + W \text{tg}_i)$ for the exponent of the signature returned in query *i*, and expand

$$\begin{aligned} S &= \sum_k a_k T^k + \sum_i b_i R_i + \sum_i c_i P_i, & \Pi &= \sum_k a'_k T^k + \sum_i b'_i R_i + \sum_i c'_i P_i, \\ R &= \sum_k a''_k T^k + \sum_i b''_i R_i + \sum_i c''_i P_i. \end{aligned}$$

The coefficients a_k and b_i vanish because no T^k or R_i monomial appears on the right-hand side of Eq. (1). The term $U(T - m^*)\Pi$ contains the monomials UTR_i , UTR_iV , and UT^{B+1} with coefficients b'_i , c'_i (through the R_iV -component of P_i), and a'_B . The first two are unique to query i , and no other term of the equation contains any of the three: in particular, every U -monomial of S comes from a component $y_i U f_{\mathbf{m}_i}(T)$ of degree at most B in T . Hence $b'_i = c'_i = a'_B = 0$ and $\Pi = Q(T)$ for a polynomial $Q(T) = \sum_{k < B} a'_k T^k$ of degree at most $B - 1$. The term $R(V + W \text{tg}^*)$ contains the monomials $T^k V$ and $R_i V^2$ with coefficients a''_k and c''_i (the latter unique to query i), and neither appears in any other term. Hence $a''_k = c''_i = 0$ and $R = \sum_i b''_i R_i$. Thus Eq. (1) simplifies to

$$\sum_i c_i (A + y_i U f_{\mathbf{m}_i}(T) + R_i (V + W \text{tg}_i)) = A + \sum_i b''_i R_i (V + W \text{tg}^*) + U(T - m^*)Q(T).$$

Comparing the coefficient of A gives $\sum_i c_i = 1$. Comparing the $R_i V$ coefficients gives $b''_i = c_i$, and comparing the $R_i W$ coefficients gives $c_i \text{tg}_i = c_i \text{tg}^*$. Hence every nonzero c_i comes from a query with $\text{tg}_i = \text{tg}^*$. If no such query exists, this contradicts $\sum_i c_i = 1$. Otherwise, the label has a single query (say i^*), so $c_{i^*} = 1$. Comparing the monomials UT^k gives

$$y_{i^*} f_{\mathbf{m}_{i^*}}(T) = (T - m^*)Q(T).$$

Since $y_{i^*} \neq 0$, this implies $f_{\mathbf{m}_{i^*}}(m^*) = 0$, contradicting $m^* \notin Q_{\text{msg}}$. $\square$

Remark 6. One can prove a weaker selective security guarantee where the adversary first chooses the label and message on which forgery will occur using the falsifiable B -BDHE exponent variant from [GWWW25]. However we focus on the stronger notion where the adversary outputs a forgery after arbitrarily many adaptively chosen signing queries and thus prove security in the GGM.

The above construction is insecure if the signer issues more than one signature for the same batch label tg . [GWWW25] construct a variant of the above construction that supports issuing multiple signatures (say q of them) for a single batch label tg but this comes at the cost of a larger public key $O(q + B)$, larger secret key $O(q)$, and a larger ciphertext size $O(q)$. Alternatively, this can be constructed generically in all the schemes by sampling q different public keys and using the same crs . The committee can then sign multiple batches using different secret keys for the same batch label. In Section 7.4, we present a construction that supports issuing multiple signatures for a single batch label tg with a larger $O(qB)$ CRS (the verification key remains constant-size) but constant sized ciphertexts.

7.3 FPTX25

Construction 5. • Setup($1^\lambda, B, q$). Sample $\tau \xleftarrow{\$} \mathbb{F}$, $\{\kappa_s\}_{s \in [q]} \xleftarrow{\$} \mathbb{F}^q$, and an independent element $g' \xleftarrow{\$} \mathbb{G}_1$.
Output

$$\text{crs} = (g, g', \{g^{\kappa_s \tau^k}\}_{s \in [q], 0 \leq k \leq B}, h, h^\tau).$$

• KeyGen(crs). Sample $\alpha \xleftarrow{\$} \mathbb{F}$. Set $\text{pk} = h^\alpha$ and $\text{pk}^\tau = h^{\alpha\tau}$. Output

$$\text{vk} = (\text{pk}, \text{pk}^\tau), \quad \text{sk} = \alpha, \quad \text{st}_0 = 0.$$

• Sign($\text{crs}, \text{vk}, \text{sk}, \mathbf{m}, \text{st}$). Parse $\text{sk} = \alpha$, $\text{vk} = (\text{pk}, \text{pk}^\tau)$, and $\text{st} = c$. If $c = q$, output $\perp$. Otherwise set $s = c + 1$, compute $\text{com}_s = g^{\kappa_s f_{\mathbf{m}}(\tau)}$ from the s -th CRS slot, and output

$$(\sigma = (s, (g' \cdot \text{com}_s)^\alpha), \quad \text{st}' = c + 1).$$

• Open($\text{crs}, \text{vk}, \sigma, \mathbf{m}$). Parse $\sigma = (s, \sigma_1)$. For each $j \in [B]$, compute $q_j(X) = f_{\mathbf{m}}(X)/(X - m_j)$ and

$$\pi_j = g^{\kappa_s q_j(\tau)}$$

from the s -th CRS slot. Output π_j .

- Verify(crs, vk, m, σ, π). Parse

$$\text{vk} = (\text{pk}, \text{pk}^\tau), \quad \sigma = (s, \sigma_1)$$

If $s \notin [q]$, output 0. Accept iff

$$e(\sigma_1, h) \stackrel{?}{=} e(g', \text{pk}) \cdot e(\pi, \text{pk}^{\tau-m}).$$

Theorem 5. In the generic bilinear-group model of Definition 1, Construction 5 is correct and q -EUF-CMA secure.

Proof. Correctness. We write $g' = g^\beta$ for the analysis. On the s -th signing query for $\mathbf{m} = (m_1, \dots, m_B)$, the signer outputs

$$\sigma_1 = g^{\alpha(\beta + \kappa_s f_{\mathbf{m}}(\tau))}.$$

For an opening at m_j , write $q_j(X) = f_{\mathbf{m}}(X)/(X - m_j)$, so $\pi_j = g^{\kappa_s q_j(\tau)}$. Then

$$e(g', \text{pk}) \cdot e(\pi_j, \text{pk}^{\tau-m_j}) = e(g, h)^{\alpha\beta + \alpha\kappa_s q_j(\tau)(\tau-m_j)} = e(\sigma_1, h).$$

q -EUF-CMA security. Let $A, T, U, \{K_s\}_{s \in [q]}$ be formal variables for $\alpha, \tau, \log_g(g'), \{\kappa_s\}_{s \in [q]}$. Let $(\mathbf{m}_i, s_i)$ be the successful signing queries. Every $\mathbb{G}_1$ exponent lies in the span of

$$1, \quad U, \quad \{K_s T^k\}_{s \in [q], 0 \leq k \leq B}, \quad \{A(U + K_{s_i} f_{\mathbf{m}_i}(T))\}_i,$$

except with the usual formal-polynomial collision probability. Suppose an accepting forgery is (m^*, σ^*, π^*) with $\sigma^* = (s^*, \sigma_1^*)$, and write the formal exponents of σ_1^*, π^* as S, Π . Verification demands that

$$S = AU + A(T - m^*)\Pi.$$

Since S and Π are $\mathbb{G}_1$ exponents, they lie in the span displayed above. Write

$$\begin{aligned} S &= a_1 + a_U U + \sum_{s,k} a_{s,k} K_s T^k + \sum_i c_i A(U + K_{s_i} f_{\mathbf{m}_i}(T)), \\ \Pi &= a'_1 + a'_U U + \sum_{s,k} a'_{s,k} K_s T^k + \sum_i c'_i A(U + K_{s_i} f_{\mathbf{m}_i}(T)). \end{aligned}$$

The monomials $AT, ATU, A^2 K_{s_i} T^{B+1}$, and $AK_s T^{B+1}$ appear in $AU + A(T - m^*)\Pi$ with coefficients a'_1, a'_U, c'_i , and $a'_{s,B}$ respectively, but no element of the span contains any of them, so they cannot appear in S . Hence $a'_1 = a'_U = c'_i = a'_{s,B} = 0$ and $\Pi = \sum_{s \in [q]} K_s Q_s(T)$ for polynomials $Q_s(T) = \sum_{k < B} a'_{s,k} T^k$ of degree at most $B - 1$.

Every monomial on the right-hand side of the verification equation contains A , so $a_1 = a_U = a_{s,k} = 0$, and comparing the remaining terms gives

$$\sum_i c_i (U + K_{s_i} f_{\mathbf{m}_i}(T)) = U + \sum_{s \in [q]} K_s (T - m^*) Q_s(T).$$

Comparing the coefficient of U gives $\sum_i c_i = 1$. Each slot is used at most once, so comparing the coefficient of K_s gives, for the unique query i in slot s if it exists,

$$c_i f_{\mathbf{m}_i}(T) = (T - m^*) Q_s(T),$$

and gives $Q_s = 0$ if no such query exists. Since $m^* \notin Q_{\text{msg}}$, $f_{\mathbf{m}_i}(m^*) \neq 0$ for every signing query, so every $c_i = 0$. This contradicts $\sum_i c_i = 1$. $\square$

7.4 Our Construction

Construction 6. • Setup($1^\lambda, B, q$). Sample $\tau, \gamma, \delta \xleftarrow{\$} \mathbb{F}$ and $\{\kappa_s\}_{s \in [q]} \xleftarrow{\$} \mathbb{F}^q$. Output

$$\text{crs} = \left(g, \{g^{\kappa_s \tau^k}\}_{s \in [q], 0 \leq k \leq B}, g^\gamma, g^\delta, h, h^\tau, h^\gamma, h^\delta \right).$$

The exponents $\tau, \gamma, \delta, \{\kappa_s\}_s$ are discarded.

• KeyGen(crs). Sample $\alpha \xleftarrow{\$} \mathbb{F}$. Set $\text{pk} = h^\alpha$ and $\text{pk}^\tau = h^{\alpha\tau}$. Output

$$\text{vk} = (\text{pk}, \text{pk}^\tau), \quad \text{sk} = \alpha, \quad \text{st}_0 = \text{ctr},$$

where ctr is a map $\mathbb{F} \rightarrow \mathbb{N}$ that tracks the number of queries made for each batch label. It is initially empty and batch labels are only added when at least one query has been made.

• Sign(crs, vk, sk, **m**, st, tg). Parse $\text{sk} = \alpha$ and $\text{st} = \text{ctr}$.

- if $\text{tg} \notin \text{dom}(\text{ctr})$, set $\text{ctr}[\text{tg}] \leftarrow 1$,
- else if $\text{ctr}[\text{tg}] = q$, output $\perp$,
- else set $\text{ctr}[\text{tg}] \leftarrow \text{ctr}[\text{tg}] + 1$.

Set $s = \text{ctr}[\text{tg}]$. Sample $\rho \xleftarrow{\$} \mathbb{F}$, compute $\text{com}_s = g^{\kappa_s f_{\mathbf{m}}(\tau)}$ from the s -th CRS slot, and output

$$\left(\sigma = \left(s, \sigma_0 = g^\rho, \sigma_1 = g^{\alpha(1+\kappa_s f_{\mathbf{m}}(\tau)) + \rho(\gamma + \delta \text{tg})} \right), \quad \text{st}' = \text{ctr} \right).$$

• Open(crs, vk, σ , **m**). Parse $\sigma = (s, \sigma_0, \sigma_1)$. If $s \notin [q]$, output $\perp$. Otherwise, for each $j \in [B]$, compute $q_j(X) = f_{\mathbf{m}}(X)/(X - m_j)$ and output

$$\pi_j = g^{\kappa_s q_j(\tau)}.$$

• Verify(crs, vk, $m, \sigma, \pi, \text{tg}$). Parse

$$\text{crs} = \left(g, \{g^{\kappa_s \tau^k}\}_{s \in [q], 0 \leq k \leq B}, g^\gamma, g^\delta, h, h^\tau, h^\gamma, h^\delta \right), \quad \text{vk} = (\text{pk}, \text{pk}^\tau) \quad \text{and} \quad \sigma = (s, \sigma_0, \sigma_1), \quad \pi = \omega.$$

If $s \notin [q]$, output 0. Otherwise check the succinct equation

$$e(\sigma_1, h) \stackrel{?}{=} e(g, \text{pk}) \cdot e(\omega, \text{pk}^{\tau-m}) \cdot e(\sigma_0, h^\gamma (h^\delta)^{\text{tg}}),$$

where $\text{pk}^{\tau-m} := \text{pk}^\tau \cdot \text{pk}^{-m}$.

Theorem 6. In the generic bilinear-group model of Definition 1, Construction 6 is correct and labeled q -EUF-CMA secure.

Proof. **Correctness.** On a signing query (tg, **m**) in slot s , the signer outputs

$$\sigma_1 = g^{\alpha(1+\kappa_s f_{\mathbf{m}}(\tau)) + \rho(\gamma + \delta \text{tg})}.$$

For an opening at m_j , write $q_j(X) = f_{\mathbf{m}}(X)/(X - m_j)$, so $\pi_j = g^{\kappa_s q_j(\tau)}$. Then

$$e(g, \text{pk}) \cdot e(\pi_j, \text{pk}^{\tau-m_j}) \cdot e(\sigma_0, h^\gamma (h^\delta)^{\text{tg}}) = e(g, h)^{\alpha + \kappa_s q_j(\tau)(\tau - m_j) + \rho(\gamma + \delta \text{tg})} = e(\sigma_1, h).$$

Labeled q -EUF-CMA security. Let $A, T, V, D, \{K_s\}_{s \in [q]}$ be formal variables for $\alpha, \tau, \gamma, \delta, \{\kappa_s\}_{s \in [q]}$. At the time of the forgery, let $(\mathbf{tg}_i, \mathbf{m}_i, s_i)$ be all successful signing queries, and let R_i be the formal randomizer used in query i . Every $\mathbb{G}_1$ exponent available to the adversary lies in the span of

$$1, \quad V, \quad D, \quad \{K_s T^k\}_{s \in [q], 0 \leq k \leq B}, \quad \{R_i\}_i, \quad \{A(1 + K_{s_i} f_{\mathbf{m}_i}(T)) + R_i(V + D \mathbf{tg}_i)\}_i,$$

except with the usual formal-polynomial collision probability.

Suppose an accepting forgery is $(\mathbf{tg}^*, m^*, \sigma^*, \pi^*)$ with $\sigma^* = (s^*, \sigma_0^*, \sigma_1^*)$. Write the formal exponents of $\sigma_0^*, \sigma_1^*, \pi^*$ as R, S, Π . Verification gives

$$S = A + A(T - m^*)\Pi + R(V + D \mathbf{tg}^*).$$

Since S, R , and Π are $\mathbb{G}_1$ exponents, they lie in the span displayed above. Write $P_i := A(1 + K_{s_i} f_{\mathbf{m}_i}(T)) + R_i(V + D \mathbf{tg}_i)$ for the exponent of the signature returned in query i , and expand

$$\begin{aligned} S &= a_1 + a_V V + a_D D + \sum_{s,k} a_{s,k} K_s T^k + \sum_i b_i R_i + \sum_i c_i P_i, \\ \Pi &= a'_1 + a'_V V + a'_D D + \sum_{s,k} a'_{s,k} K_s T^k + \sum_i b'_i R_i + \sum_i c'_i P_i, \\ R &= a''_1 + a''_V V + a''_D D + \sum_{s,k} a''_{s,k} K_s T^k + \sum_i b''_i R_i + \sum_i c''_i P_i. \end{aligned}$$

The term $A(T - m^*)\Pi$ contains the monomials $AT, ATV, ATD, ATR_i, ATR_i V$, and $AK_s T^{B+1}$ with coefficients $a'_1, a'_V, a'_D, b'_i, c'_i$ (through the $R_i V$ -component of P_i , and unique to query i since R_i is), and $a'_{s,k}$. None of these monomials appears in any other term of the verification equation (the P_i contain the monomials A and $AK_s T^k$, but never AT). Hence all of these coefficients vanish and $\Pi = \sum_{s \in [q]} K_s Q_s(T)$ for polynomials $Q_s(T) = \sum_{k < B} a'_{s,k} T^k$ of degree at most $B - 1$. We now compare coefficients on the two sides of the verification equation.

- *Monomials $R_i V^2, V^2, VD, D^2, K_s T^k V, K_s T^k D$.* None of these occur on the left side or in $A(T - m^*)\Pi$. On the right, the term $c''_i P_i(V + D \mathbf{tg}^*)$ contributes the monomial $R_i V^2$ with coefficient exactly c''_i (through the $R_i V$ -component of P_i); since $R_i V^2$ is unique to query i and appears in no other term, $c''_i = 0$ for every i . The coefficients a''_V, a''_D , and $a''_{s,k}$ likewise contribute the monomials V^2, VD , and $K_s T^k V$, which appear nowhere else, so they vanish as well and $R = a''_1 + \sum_i b''_i R_i$.
- *Monomials $R_i V$ and $R_i D$.* Fix a query i . On the left side, the only source of these monomials is the term $c_i P_i$, whose component $R_i(V + D \mathbf{tg}_i)$ contributes $R_i V$ with coefficient c_i and $R_i D$ with coefficient $c_i \mathbf{tg}_i$. (The term $b_i R_i$ contributes only the bare monomial R_i , and the A -components of the P_i contain no R_i .) On the right side, $A(T - m^*)\Pi$ is free of both monomials, since $\Pi = \sum_s K_s Q_s(T)$ by the elimination above, so the only source is $R(V + D \mathbf{tg}^*)$. Substituting $R = a''_1 + \sum_i b''_i R_i$ from the previous item, it contributes $R_i V$ with coefficient b''_i and $R_i D$ with coefficient $b''_i \mathbf{tg}^*$. Comparing coefficients of the two monomials gives

$$c_i = b''_i \quad \text{and} \quad c_i \mathbf{tg}_i = b''_i \mathbf{tg}^*,$$

and substituting the first equation into the second yields $c_i(\mathbf{tg}_i - \mathbf{tg}^*) = 0$. Hence every query with $\mathbf{tg}_i \neq \mathbf{tg}^*$ has $c_i = 0$: only signatures issued under the forgery label can contribute to the forged signature.

- *Monomial A .* The left side carries $\sum_i c_i$ and the right side carries 1, so $\sum_i c_i = 1$.
- *Monomials $AK_s T^k$.* By the second item, only queries with $\mathbf{tg}_i = \mathbf{tg}^*$ contribute, and for a fixed label each slot is used at most once. Comparing the K_s -block for each slot s therefore gives, for the unique query i with $(\mathbf{tg}_i, s_i) = (\mathbf{tg}^*, s)$ if it exists,

$$c_i f_{\mathbf{m}_i}(T) = (T - m^*)Q_s(T),$$

and gives $Q_s = 0$ if no such query exists.

Since $m^* \notin Q_{\text{msg}}$, $f_{\mathbf{m}_i}(m^*) \neq 0$ for every signing query, so the last item forces every such $c_i = 0$. Thus all $c_i = 0$, contradicting $\sum_i c_i = 1$. $\square$

Threshold Signing. Our scheme also supports threshold signing by Shamir secret sharing α and having the signers sample ρ independently. For security, the signers must agree on the slot index s and use each slot for at most one batch per label. We sketch the construction below. Shamir secret shares $(\alpha_i)_{i \in [n]}$ of α are distributed to n parties. Concretely, party i with share α_i samples ρ_i and outputs

$$\text{psig}_i = \left(s, \sigma_{i,0} = g^{\rho_i}, \sigma_{i,1} = g^{\alpha_i(1+\kappa_s f_{\mathbf{m}}(\tau)) + \rho_i(\gamma + \delta \mathbf{t}\mathbf{g})} \right).$$

Given shares from a set S of t parties, the signature can be recovered using Lagrange interpolation with coefficients λ_i^S :

$$\begin{aligned} \sigma_0 &= \prod_{i \in S} \sigma_{i,0}^{\lambda_i^S} = g^{\tilde{\rho}}, \quad \text{where} \quad \tilde{\rho} = \sum_{i \in S} \lambda_i^S \rho_i, \\ \sigma_1 &= \prod_{i \in S} \sigma_{i,1}^{\lambda_i^S} = g^{(\sum_{i \in S} \lambda_i^S \alpha_i)(1+\kappa_s f_{\mathbf{m}}(\tau)) + \tilde{\rho}(\gamma + \delta \mathbf{t}\mathbf{g})} = g^{\alpha(1+\kappa_s f_{\mathbf{m}}(\tau)) + \tilde{\rho}(\gamma + \delta \mathbf{t}\mathbf{g})}, \end{aligned}$$

using $\sum_{i \in S} \lambda_i^S \alpha_i = \alpha$. The recovered signature is equivalent to a single-signer signature with randomizer $\tilde{\rho}$, and still passes verification: for the opening $\pi_j = g^{\kappa_s q_j(\tau)}$ produced by Open,

$$e(g, \text{pk}) \cdot e(\pi_j, \text{pk}^{\tau - m_j}) \cdot e(\sigma_0, h^\gamma (h^\delta)^{\mathbf{t}\mathbf{g}}) = e(g, h)^{\alpha + \alpha \kappa_s q_j(\tau)(\tau - m_j) + \tilde{\rho}(\gamma + \delta \mathbf{t}\mathbf{g})} = e(\sigma_1, h).$$

8 Acknowledgments

We thank Patrick O’Grady and Lucas Meier for their valuable feedback.

References

- [ABD⁺25] Amit Agarwal, Kushal Babel, Sourav Das, Babak Poorebrahim Gilkalaye, Arup Mondal, Benny Pinkas, Peter Rindal, and Aayush Yadav. Weighted batched threshold encryption with applications to mempool privacy. Cryptology ePrint Archive, Report 2025/2115, 2025. [1](#), [4](#)
- [ADG⁺26] Amit Agarwal, Sourav Das, Babak Poorebrahim Gilkalaye, Peter Rindal, and Victor Shoup. BTX: Simple and efficient batch threshold encryption. Cryptology ePrint Archive, Report 2026/754, 2026. [1](#), [4](#)
- [AFP25] Amit Agarwal, Rex Fernando, and Benny Pinkas. Efficiently-thresholdizable batched identity based encryption, with applications. In Yael Tauman Kalai and Seny F. Kamara, editors, *CRYPTO 2025, Part III*, volume 16002 of *LNCS*, pages 69–100. Springer, Cham, August 2025. [1](#), [2](#), [3](#), [4](#), [5](#), [7](#), [9](#), [17](#)
- [AL11] Nuttapong Attrapadung and Benoît Libert. Homomorphic network coding signatures in the standard model. In Dario Catalano, Nelly Fazio, Rosario Gennaro, and Antonio Nicolosi, editors, *PKC 2011*, volume 6571 of *LNCS*, pages 17–34. Springer, Berlin, Heidelberg, March 2011. [7](#)
- [BB04a] Dan Boneh and Xavier Boyen. Efficient selective-ID secure identity based encryption without random oracles. In Christian Cachin and Jan Camenisch, editors, *EUROCRYPT 2004*, volume 3027 of *LNCS*, pages 223–238. Springer, Berlin, Heidelberg, May 2004. [6](#)
- [BB04b] Dan Boneh and Xavier Boyen. Short signatures without random oracles. In Christian Cachin and Jan Camenisch, editors, *EUROCRYPT 2004*, volume 3027 of *LNCS*, pages 56–73. Springer, Berlin, Heidelberg, May 2004. [8](#)

- [BB08] Dan Boneh and Xavier Boyen. Short signatures without random oracles and the SDH assumption in bilinear groups. *Journal of Cryptology*, 21(2):149–177, April 2008. [8](#)
- [BBS04] Dan Boneh, Xavier Boyen, and Hovav Shacham. Short group signatures. In Matthew Franklin, editor, *CRYPTO 2004*, volume 3152 of *LNCS*, pages 41–55. Springer, Berlin, Heidelberg, August 2004. [8](#)
- [BC16] Olivier Blazy and Céline Chevalier. Structure-preserving smooth projective hashing. In Jung Hee Cheon and Tsuyoshi Takagi, editors, *ASIACRYPT 2016, Part II*, volume 10032 of *LNCS*, pages 339–369. Springer, Berlin, Heidelberg, December 2016. [2](#), [3](#), [29](#)
- [BCF⁺25] Jan Bormet, Arka Rai Choudhuri, Sebastian Faust, Sanjam Garg, Hussien Othman, Guru-Vamsi Policharla, Ziyang Qu, and Mingyuan Wang. BEAST-MEV: Batched threshold encryption with silent setup for MEV prevention. Cryptology ePrint Archive, Report 2025/1419, 2025. [1](#), [4](#)
- [BF01] Dan Boneh and Matthew K. Franklin. Identity-based encryption from the Weil pairing. In Joe Kilian, editor, *CRYPTO 2001*, volume 2139 of *LNCS*, pages 213–229. Springer, Berlin, Heidelberg, August 2001. [2](#), [3](#)
- [BF11] Dan Boneh and David Mandell Freeman. Homomorphic signatures for polynomial functions. In Kenneth G. Paterson, editor, *EUROCRYPT 2011*, volume 6632 of *LNCS*, pages 149–168. Springer, Berlin, Heidelberg, May 2011. [8](#)
- [BFKT24] Jan Bobolz, Pooya Farshim, Markulf Kohlweiss, and Akira Takahashi. The brave new world of global generic groups and UC-secure zero-overhead SNARKs. In Elette Boyle and Mohammad Mahmoody, editors, *TCC 2024, Part I*, volume 15364 of *LNCS*, pages 90–124. Springer, Cham, December 2024. [8](#)
- [BFKW09] Dan Boneh, David Freeman, Jonathan Katz, and Brent Waters. Signing a linear subspace: Signature schemes for network coding. In Stanislaw Jarecki and Gene Tsudik, editors, *PKC 2009*, volume 5443 of *LNCS*, pages 68–87. Springer, Berlin, Heidelberg, March 2009. [7](#)
- [BFOQ25] Jan Bormet, Sebastian Faust, Hussien Othman, and Ziyang Qu. BEAT-MEV: Epochless approach to batched threshold encryption for MEV prevention. In Lujo Bauer and Giancarlo Pellegrino, editors, *USENIX Security 2025*, pages 3457–3476. USENIX Association, August 2025. [1](#), [4](#)
- [BL20] Fabrice Benhamouda and Huijia Lin. Mr NISC: Multiparty reusable non-interactive secure computation. In Rafael Pass and Krzysztof Pietrzak, editors, *TCC 2020, Part II*, volume 12551 of *LNCS*, pages 349–378. Springer, Cham, November 2020. [2](#), [3](#), [29](#)
- [BLS01] Dan Boneh, Ben Lynn, and Hovav Shacham. Short signatures from the Weil pairing. In Colin Boyd, editor, *ASIACRYPT 2001*, volume 2248 of *LNCS*, pages 514–532. Springer, Berlin, Heidelberg, December 2001. [5](#), [9](#)
- [BNRT26] Dan Boneh, Rohit Nema, Arnab Roy, and Ertem Nusret Tas. Efficient batch threshold encryption using partial fraction techniques. Cryptology ePrint Archive, Report 2026/674, 2026. [1](#), [4](#)
- [CGPW25] Arka Rai Choudhuri, Sanjam Garg, Guru-Vamsi Policharla, and Mingyuan Wang. Practical mempool privacy via one-time setup batched threshold encryption. In Lujo Bauer and Giancarlo Pellegrino, editors, *USENIX Security 2025*, pages 3477–3495. USENIX Association, August 2025. [1](#), [2](#), [3](#), [4](#), [5](#), [7](#), [9](#), [17](#)
- [CL03] Jan Camenisch and Anna Lysyanskaya. A signature scheme with efficient protocols. In Stelvio Cimato, Clemente Galdi, and Giuseppe Persiano, editors, *SCN 02*, volume 2576 of *LNCS*, pages 268–289. Springer, Berlin, Heidelberg, September 2003. [8](#)

- [FK23] Dankrad Feist and Dmitry Khovratovich. Fast amortized KZG proofs. Cryptology ePrint Archive, Report 2023/033, 2023. [2](#), [17](#)
- [FN16] Dario Fiore and Anca Nitulescu. On the (in)security of SNARKs in the presence of oracles. In Martin Hirt and Adam D. Smith, editors, *TCC 2016-B, Part I*, volume 9985 of *LNCS*, pages 108–138. Springer, Berlin, Heidelberg, October / November 2016. [3](#), [13](#)
- [FPTX25] Rex Fernando, Guru-Vamsi Policharla, Andrei Tonkikh, and Zhuolun Xiang. TrX: Encrypted mempools in high performance BFT protocols. Cryptology ePrint Archive, Report 2025/2032, 2025. [1](#), [2](#), [3](#), [4](#), [5](#), [9](#), [17](#)
- [Fre12] David Mandell Freeman. Improved security for linearly homomorphic signatures: A generic framework. In Marc Fischlin, Johannes Buchmann, and Mark Manulis, editors, *PKC 2012*, volume 7293 of *LNCS*, pages 697–714. Springer, Berlin, Heidelberg, May 2012. [7](#), [8](#)
- [GHK⁺25] Sanjam Garg, Mohammad Hajiabadi, Dimitris Kolonelos, Abhiram Kothapalli, and Guru-Vamsi Policharla. A framework for witness encryption from linearly verifiable SNARKs and applications. In Yael Tauman Kalai and Seny F. Kamara, editors, *CRYPTO 2025, Part III*, volume 16002 of *LNCS*, pages 504–539. Springer, Cham, August 2025. [2](#), [3](#), [5](#), [29](#), [30](#)
- [GKKR10] Rosario Gennaro, Jonathan Katz, Hugo Krawczyk, and Tal Rabin. Secure network coding over the integers. In Phong Q. Nguyen and David Pointcheval, editors, *PKC 2010*, volume 6056 of *LNCS*, pages 142–160. Springer, Berlin, Heidelberg, May 2010. [7](#)
- [GKPW24] Sanjam Garg, Dimitris Kolonelos, Guru-Vamsi Policharla, and Mingyuan Wang. Threshold encryption with silent setup. In Leonid Reyzin and Douglas Stebila, editors, *CRYPTO 2024, Part VII*, volume 14926 of *LNCS*, pages 352–386. Springer, Cham, August 2024. [2](#), [3](#), [5](#), [7](#), [29](#)
- [GVW15] Sergey Gorbunov, Vinod Vaikuntanathan, and Daniel Wichs. Leveled fully homomorphic signatures from standard lattices. In Rocco A. Servedio and Ronitt Rubinfeld, editors, *47th ACM STOC*, pages 469–477. ACM Press, June 2015. [8](#)
- [GWWW25] Junqing Gong, Brent Waters, Hoeteck Wee, and David J. Wu. Threshold batched identity-based encryption from pairings in the plain model. Cryptology ePrint Archive, Report 2025/2103, 2025. [1](#), [2](#), [3](#), [4](#), [5](#), [6](#), [7](#), [8](#), [9](#), [17](#), [19](#), [21](#)
- [JMSW02] Robert Johnson, David Molnar, Dawn Xiaodong Song, and David Wagner. Homomorphic signature schemes. In Bart Preneel, editor, *CT-RSA 2002*, volume 2271 of *LNCS*, pages 244–262. Springer, Berlin, Heidelberg, February 2002. [7](#)
- [KZG10] Aniket Kate, Gregory M. Zaverucha, and Ian Goldberg. Constant-size commitments to polynomials and their applications. In Masayuki Abe, editor, *ASIACRYPT 2010*, volume 6477 of *LNCS*, pages 177–194. Springer, Berlin, Heidelberg, December 2010. [4](#)
- [Pol26] Guru-Vamsi Policharla. A simple batched threshold encryption scheme. Cryptology ePrint Archive, Report 2026/760, 2026. [1](#), [4](#)
- [PS16] David Pointcheval and Olivier Sanders. Short randomizable signatures. In Kazue Sako, editor, *CT-RSA 2016*, volume 9610 of *LNCS*, pages 111–126. Springer, Cham, February / March 2016. [8](#)
- [Sch80] Jacob T Schwartz. Fast probabilistic algorithms for verification of polynomial identities. *Journal of the ACM (JACM)*, 27(4):701–717, 1980. [9](#)
- [Sho97] Victor Shoup. Lower bounds for discrete logarithms and related problems. In Walter Fumy, editor, *EUROCRYPT’97*, volume 1233 of *LNCS*, pages 256–266. Springer, Berlin, Heidelberg, May 1997. [8](#)

- [TZ23] Stefano Tessaro and Chenzhi Zhu. Revisiting BBS signatures. In Carmit Hazay and Martijn Stam, editors, *EUROCRYPT 2023, Part V*, volume 14008 of *LNCS*, pages 691–721. Springer, Cham, April 2023. [8](#)
- [Wat05] Brent R. Waters. Efficient identity-based encryption without random oracles. In Ronald Cramer, editor, *EUROCRYPT 2005*, volume 3494 of *LNCS*, pages 114–127. Springer, Berlin, Heidelberg, May 2005. [8](#)
- [Zip79] Richard Zippel. Probabilistic algorithms for sparse polynomials. In *International symposium on symbolic and algebraic manipulation*, pages 216–226. Springer, 1979. [9](#)

A Our Batched IBE Scheme

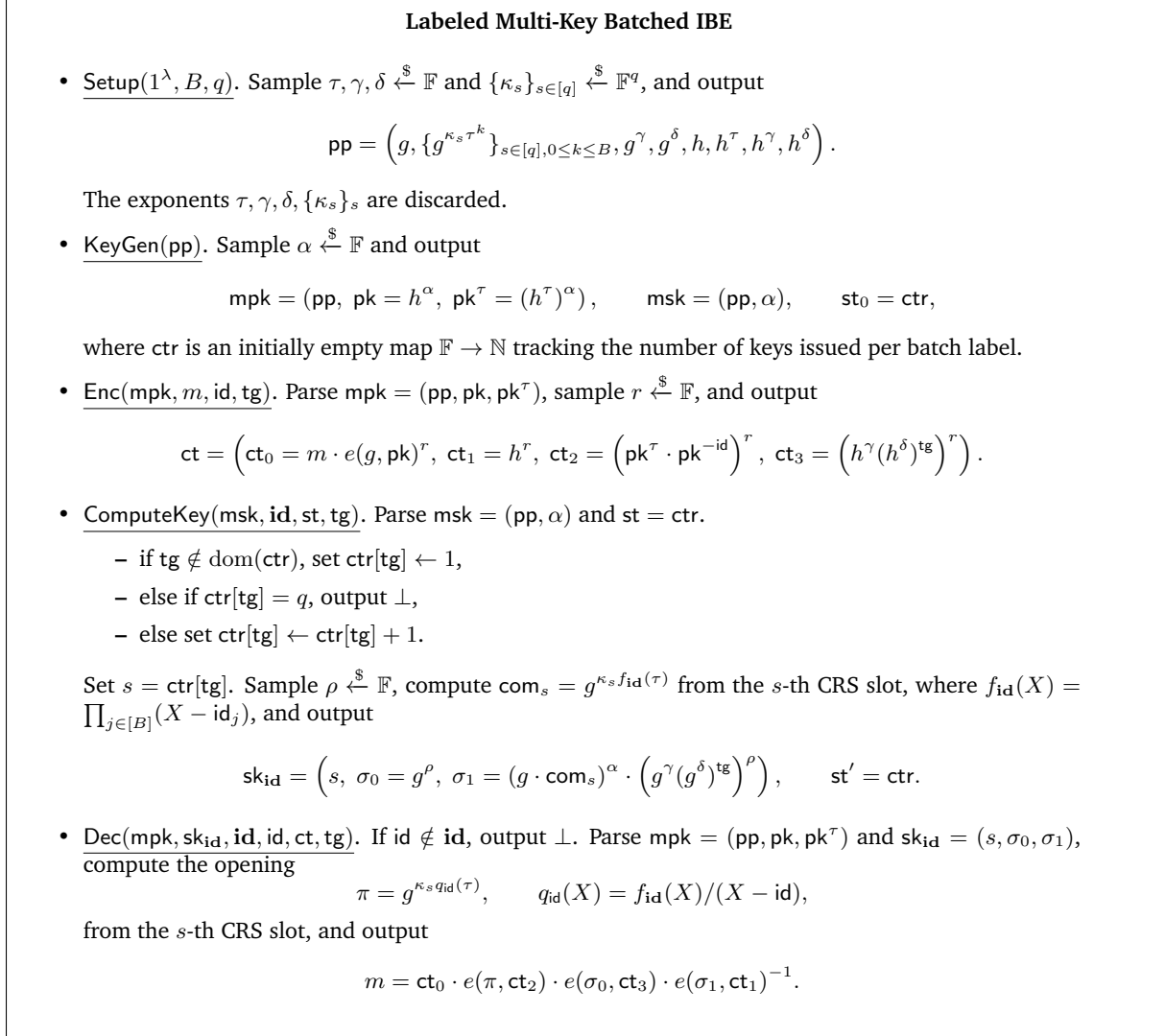


Figure 1: Our Batched IBE scheme

Corollary 1. *In the generic bilinear-group model, the batched IBE scheme of Fig. 1, with message space $\mathcal{M} = \mathbb{G}_T$ and identity and batch-label spaces $\mathcal{I} = \mathcal{T} = \mathbb{F}$, is correct and labeled q -adaptively secure (Definition 10).*

Proof sketch. The scheme of Fig. 1 is Construction 1 instantiated with our multi-message signature scheme (Construction 6) and the witness encryption of [BC16, BL20, GKPW24, GHK⁺25] inlined in the Enc and Dec algorithms, so the claim follows from Theorem 1 together with Theorem 6, provided the witness encryption is extractably secure in the presence of the signing-oracle family $\mathbb{O}[\Pi_{\text{MMS}}]$.

Our verification algorithm is deterministic, so $\mathcal{R}_{\text{MMS}}$ is an NP relation consisting of a single pairing-product equation that is linear in the witness $(\sigma_0, \sigma_1, \pi)$. For such relations, the witness encryption above satisfies perfect correctness and is extractably secure in the generic group model [GHK⁺25], with a straight-line extractor that observes the adversary's group-oracle queries. Note that this remains true even in the

presence of the signing oracle $\mathbb{O}[\Pi_{\text{MMS}}]$ (Remark 3) as the extractor in [GHK⁺25] only relies on the encryption randomness appearing exclusively in the challenge ciphertext to successfully extract the witness. Signing queries made after the challenge may depend on the ciphertext, but the signing oracle in all the schemes we consider only accepts field elements ($\mathcal{I} = \mathcal{T} = \mathbb{F}$) and answers using its own key material and coins, so the encryption randomness never enters its responses and the extraction is unaffected. If instead we consider a signing oracle that accepts group elements, then a more careful treatment is needed to show extractable security in the presence of signing oracles. $\square$